

嵌入式系统

第二讲 ARM体系结构

教师：郭玉臣

Mail: yuchenguo@tongji.edu.cn



内 容

- 一、计算机体系结构
- 二、**ARM**系统结构概览
- 三、**ARM**编程模型



一、计算机体系结构

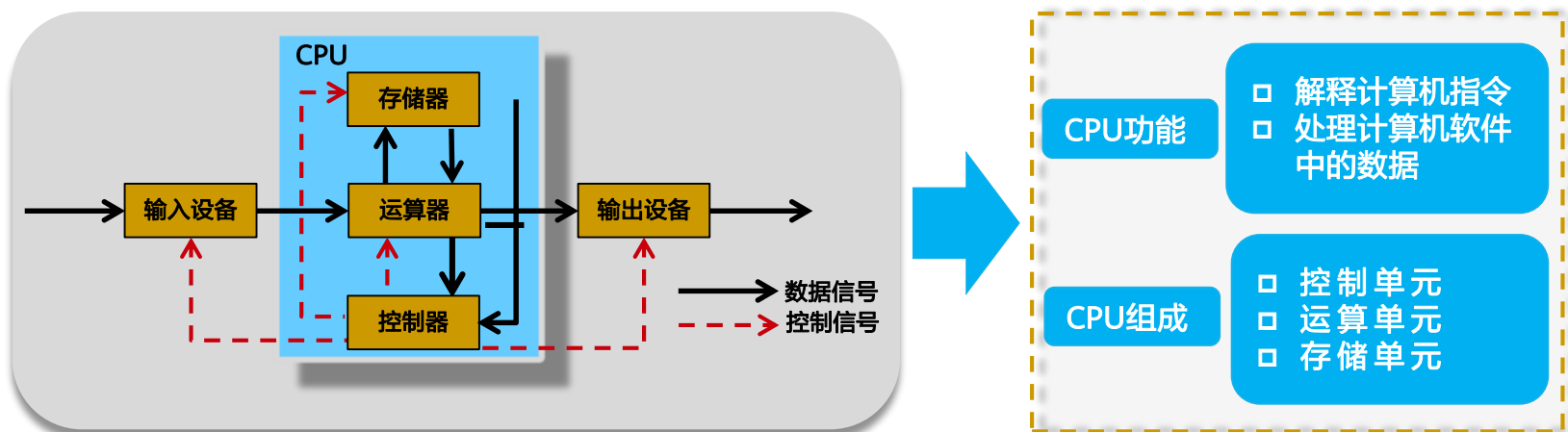
1、系统结构概述

2、四大CPU体系结构

- CISC: X86 (Intel AMD)
- RISC: ARM、MIPS、PowerPC

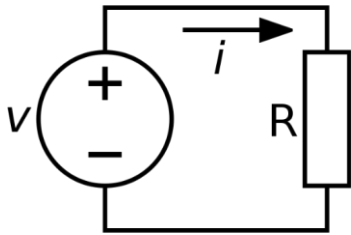
什么是中央处理器

- 中央处理器（Central Processing Unit, CPU）是计算机的主要设备之一，功能主要是解释计算机指令以及处理计算机软件中的数据；
- 中央处理器主要包括控制器，运算器，高速缓冲存储器（Cache）及实现它们之间联系的数据、控制及状态的总线；
- 它与主存（Memory）和输入/输出设备合称为电子计算机三大核心部件。



什么是芯片

- 芯片 (Chip) 是一种把电路 (主要包括半导体设备, 也包括被动组件等) 小型化的方式, 并制造在半导体晶圆表面上。



电路



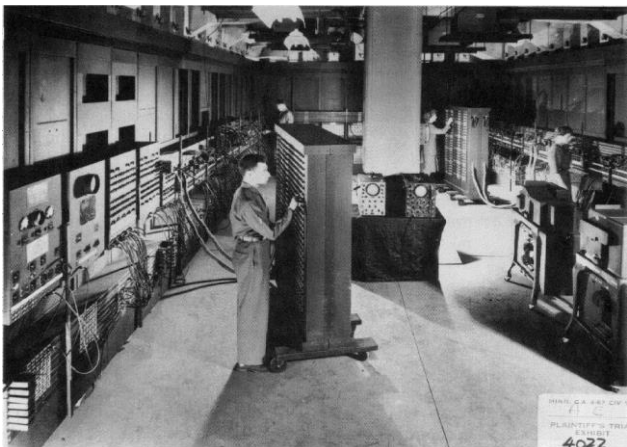
晶圆



芯片

芯片是信息社会的基石

世界第一台通用电子计算机—ENIAC



1946年诞生，由18,800多个电子管组成，重量30多吨，占地面积170多平方米。

得益于芯片的发展



小型化，高性能



内存



手机

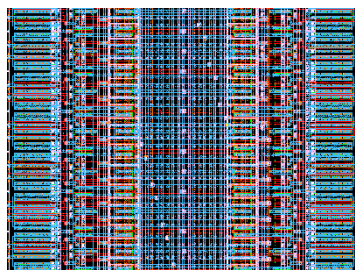


计算机



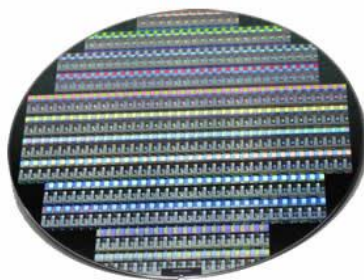
数码相机

芯片行业产业链



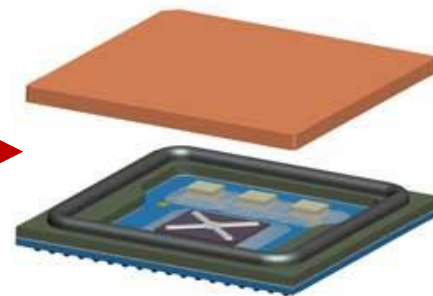
芯片设计

集成电路设计涉及对电子器件和器件间互连线模型的创建。



晶圆加工

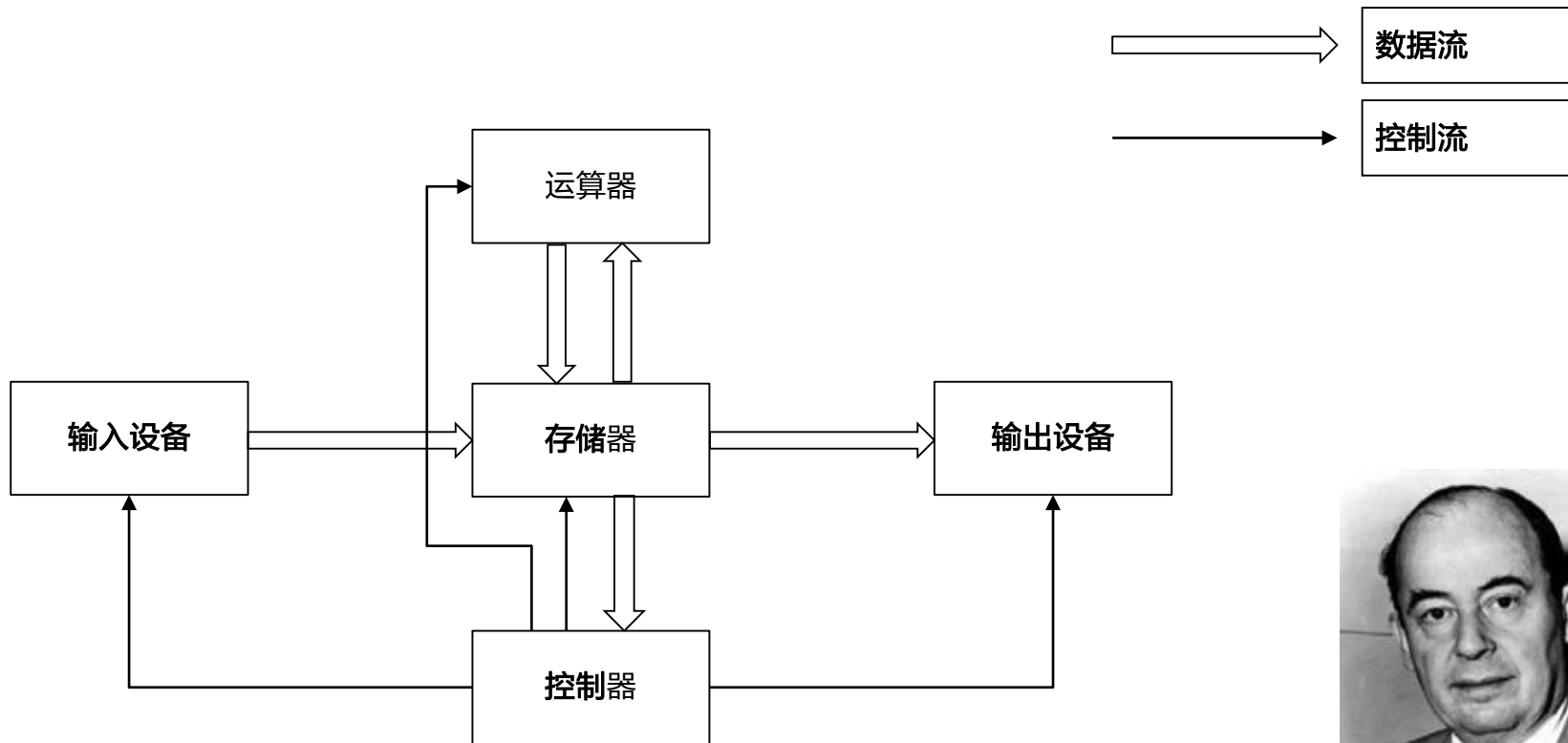
晶圆加工是一系列化学处理步骤，使得电子电路逐渐形成在使用纯半导体材料制作的芯片上。



芯片封装测试

封装是将器件的核心晶粒封装在一个支撑物之内的过程，这个封装可以防止物理损坏以及化学腐蚀，并提供对外连接的引脚，之后将进行集成电路性能测试。

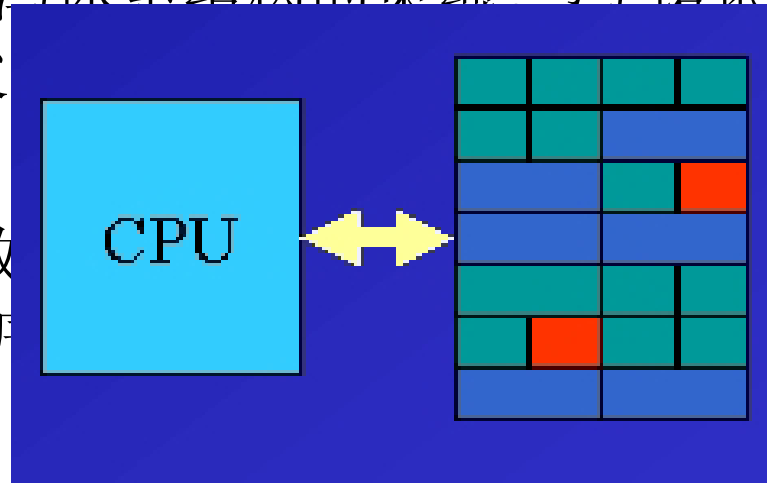
冯·诺依曼结构



计算机之父——约翰·冯·诺依曼

--冯.诺依曼结构 (von Neumann architecture)

- 冯.诺依曼体系结构是现代计算机的基础，现在大多计算机仍是冯.诺依曼计算机的组织结构，只是作了一些改进而已，并没有从根本上突破冯体系结构的束缚。冯.诺依曼也因此被人们称为“计算机之父”。
- 五大部件：ALU，
- 程序指令和数据存放在同一地址空间，CPU通过同一条总线访问程序的。
- 存储程序(stored program)：程序以数字形式存在，可以与数据一样被读写。
- <http://baike.baidu.com/view/9427882.htm?fromtitle=%E5%86%AF%E8%AF%BA%E4%BE%9D%E6%9B%BC%E4%BD%93%E7%B3%BB%E7%BB%93%E6%9E%84&type=syn>



Output

位置，CPU通过
的宽度是相同的

由于 CPU 和存储器之间数据流量大，因此计算机的处理能力经常受到存储总线传输能力的限制。这种被称为“冯·诺依曼瓶颈”的限制，已成为常规计算机性能提高的障碍之一。

2. 作为冯·诺依曼式计算机最杰出的特点并促使其迅速发展的程序，目前也变成一个有待改善的问题。因为在计算机中指令执行的顺序由程序决定，所以程序设计必须仔细地确定课题的处理顺序。对于复杂的课题，程序设计、编码、查错等也变得更为复杂。这些情况引起了所谓的“冯·诺依曼式计算机软件危机”。

3. 硬件资源未能充分利用，尤其是高速处理机更是如此。例如一个由512台处理机构成的多处理机系统，其硬件空闲时间高达15%以上。

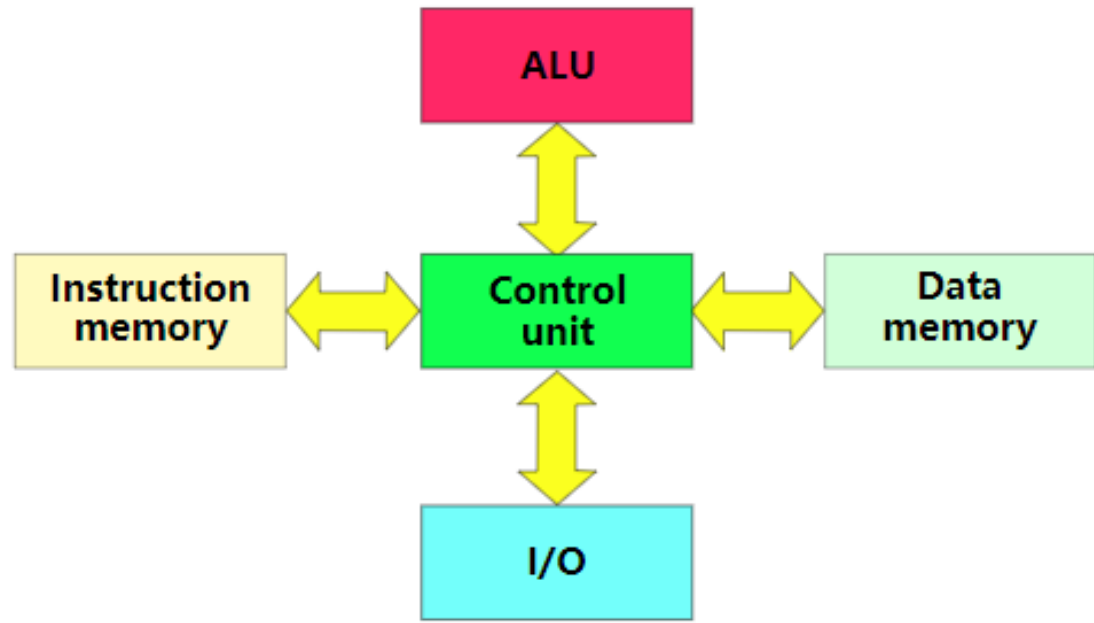
4. 面向数值计算的传统机器的体系结构，不适宜非数值信息（文件、图形、图象、程序等）的处理。因而遇到非数值操作，例如排序、检索、更新、插入、删除、演绎、匹配等，只能通过执行相应的程序来实现，故效率较低。

5. 冯·诺依曼式计算机控制结构的顺序性，不能实现真正的并行处理。在传统机器中，并行的概念受到限制。所谓的并行处理主要体现在一个处理机中不同进程的切换，或由程序员将程序分解成并行的指令流或数据流，由各个处理机执行。

6. 传统计算机难以处理自然语言。目前作为用户与计算机接口界面软件的设计语言，和人们习惯使用的自然语言之间还有相当距离，这对改善人机联系和计算机的广泛应用带来困难。

7. 冯·诺依曼式计算机具有的简单的逻辑运算或逻辑判断功能，远不能适应复杂的问题求解与推理的要求。说明这种体系结构本身不具备归纳、演绎的推理功能。

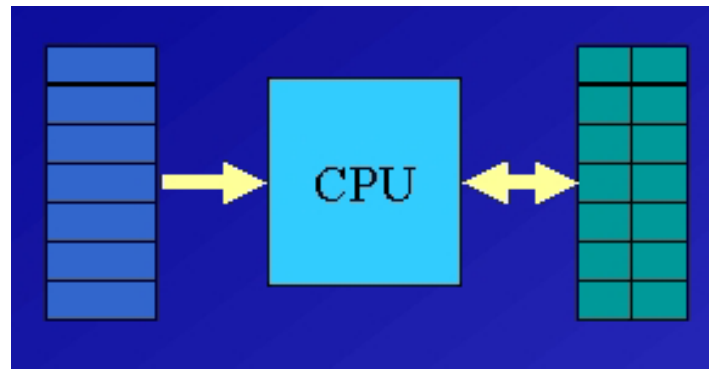
哈佛结构



程序指令存储和数据存储分开的存储器结构，主要特点：将程序和数据存储在不同的存储空间中，即程序存储器和数据存储器是两个独立的存储器，每个存储器独立编址、独立访问，目的是为了减轻程序运行时的访存瓶颈。哈佛架构的中央处理器典型代表ARM9/10及后续ARMv8的处理器，例如：华为鲲鹏920处理器。

--哈佛体系结构 (Harvard architecture)

- 程序与数据有单独的存储器，它们通过不同的总线来访问
- 一首先到程序指令存储器中读取程序指令，解码后得到数据地址，再到相应的数据存储器中读取数据，并执行。在执行一条指令的同时可以预先读取下一条指令
- 一指令和数据可以有不同的数据宽度(如PIC的程序指令是14位，而数据是8位)

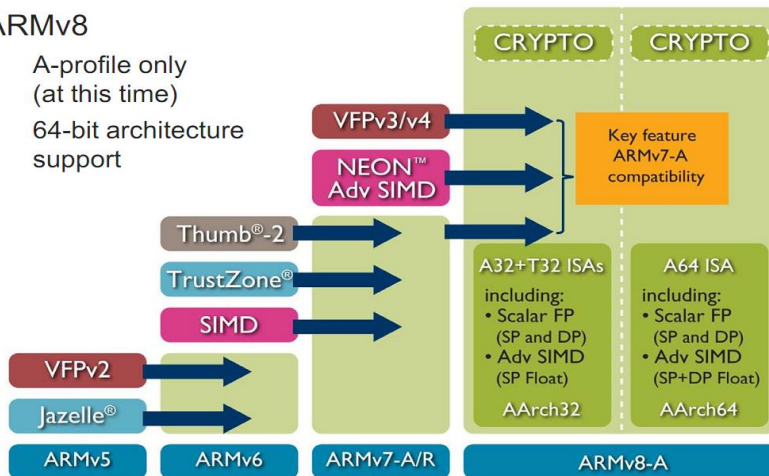


指令集：RISC vs CISC

- ARM：使用精简指令集（RISC），大幅简化架构，仅保留所需要的指令，可以让整个处理器更为简化，拥有小体积、高效能的特性。ARMv8架构支持64位操作，指令32位，寄存器64位，寻址能力64位。指令集使用NEON扩展结构；
- X86：使用复杂指令集（CISC），以增加处理器本身复杂度为代价，换取更高的性能。X86指令集从MMX，发展到了SSE，AVX；

ARMv8

- A-profile only (at this time)
- 64-bit architecture support



ARMv8架构的特点：

- 31个64位通用寄存器，原来架构只有15个通用寄存器；
- 新指令集支持64位运算，指令中的寄存器编码由4位扩充到5位；
- 新指令集仍然是32位，减少了条件执行指令，条件执行指令的4位编码释放出来用于寄存器编码；
- 堆栈指针SP和程序指针PC都不再是通用寄存器了，同时推出了零值寄存器（类似PowerPC的r0）；
- A64与A32的高级SIMD和FP相同；
- 高级SIMD与VFP共享浮点寄存器，支持128位宽的vector；
- 新增加解密指令。



复杂指令集计算机 (CISC)

■ •背景:

- 存储资源紧缺, 强调编译优化
- 增强指令功能, 设置一些功能复杂的指令以减少完成一个任务所需的指令数目
- 通过减少指令数达到提高运行数度的目的

■ •特点:

- 指令格式不固定, 指令可长可短, 操作数可多可少
- 寻址方式复杂多样, 操作数可来自寄存器, 也可来自存储器
- 使用微代码。指令集可以直接在微代码记忆体 (比主记忆体的速度快很多) 里执行
- 允许设计师实现CISC体系机器的向上相容。新的系统可以使用一个包含早期系统的指令超集合
- 微程式指令的格式与高阶语言相匹配, 因而编译器的设计较简单
- $CPI > 5$, 指令越复杂, CPI越大

■ CISC的缺陷

■ •指令使用频度不均衡:

□ “80~20”理论: 80%的计算任务就能完成

□ 扩充的复杂指令往往是低频度:

■ •大量复杂指令的控制逻辑不

□ 微程序的使用反而制约了速度: (5-10倍)

■ •CISC指令的格式长短不一, 成。

□ 执行较慢的指令, 将影响整台机器的执行效率。

□ 不利于采用先进指令级并行技术。

■ •软硬功能分配

□ —复杂指令增加硬件的复杂度, 使指令执行周期大大加长, 直接访存次数增多, 数据重复利用率低。

典型指令使用频度

指令类型	使用指令使用频度
数据传送类	43%
转/跳控制类	23%
算术运算类	15%
比较类	13%
逻辑运算类	5%
其他	1%

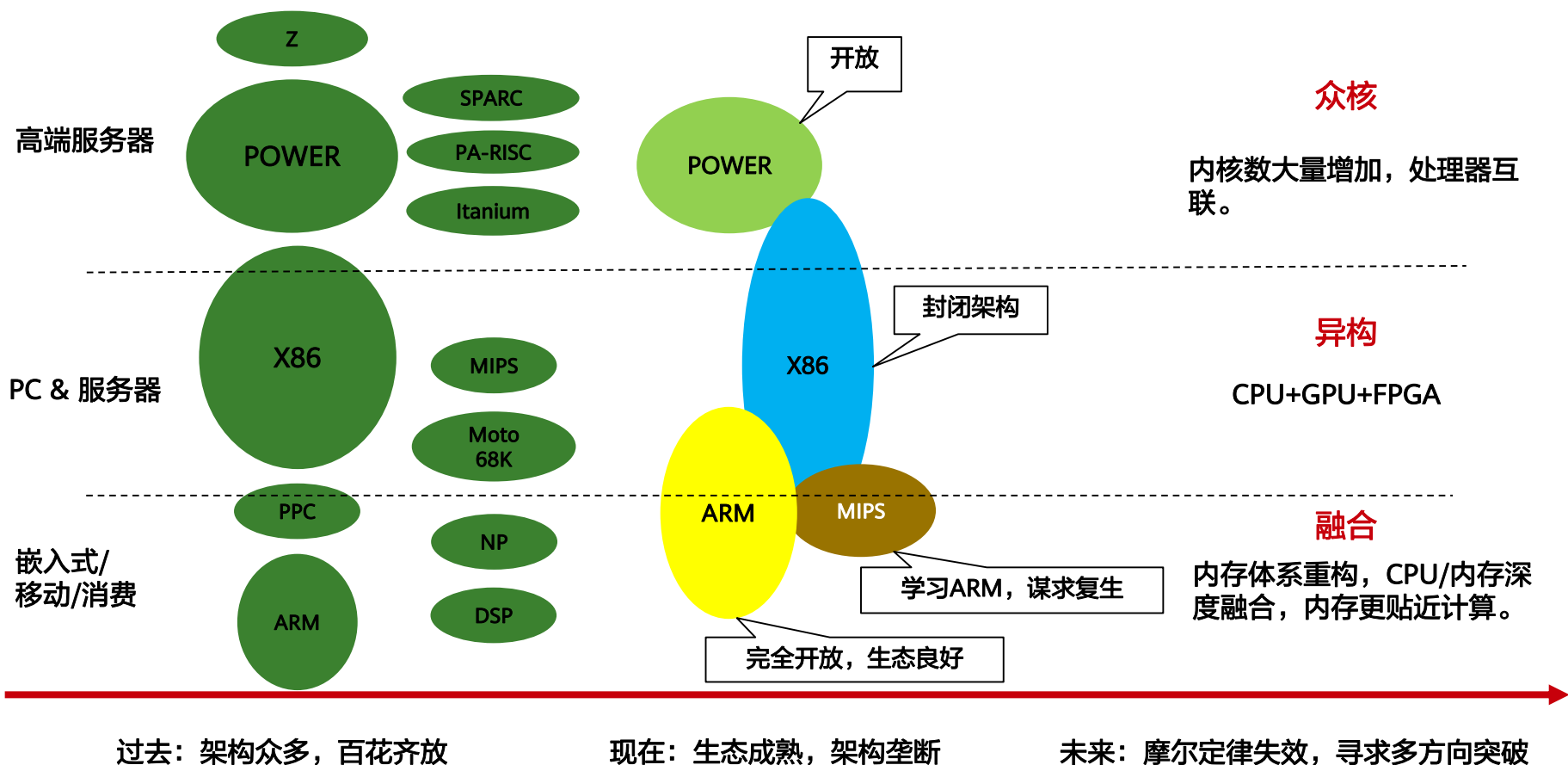


精简指令集计算机 (RISC)

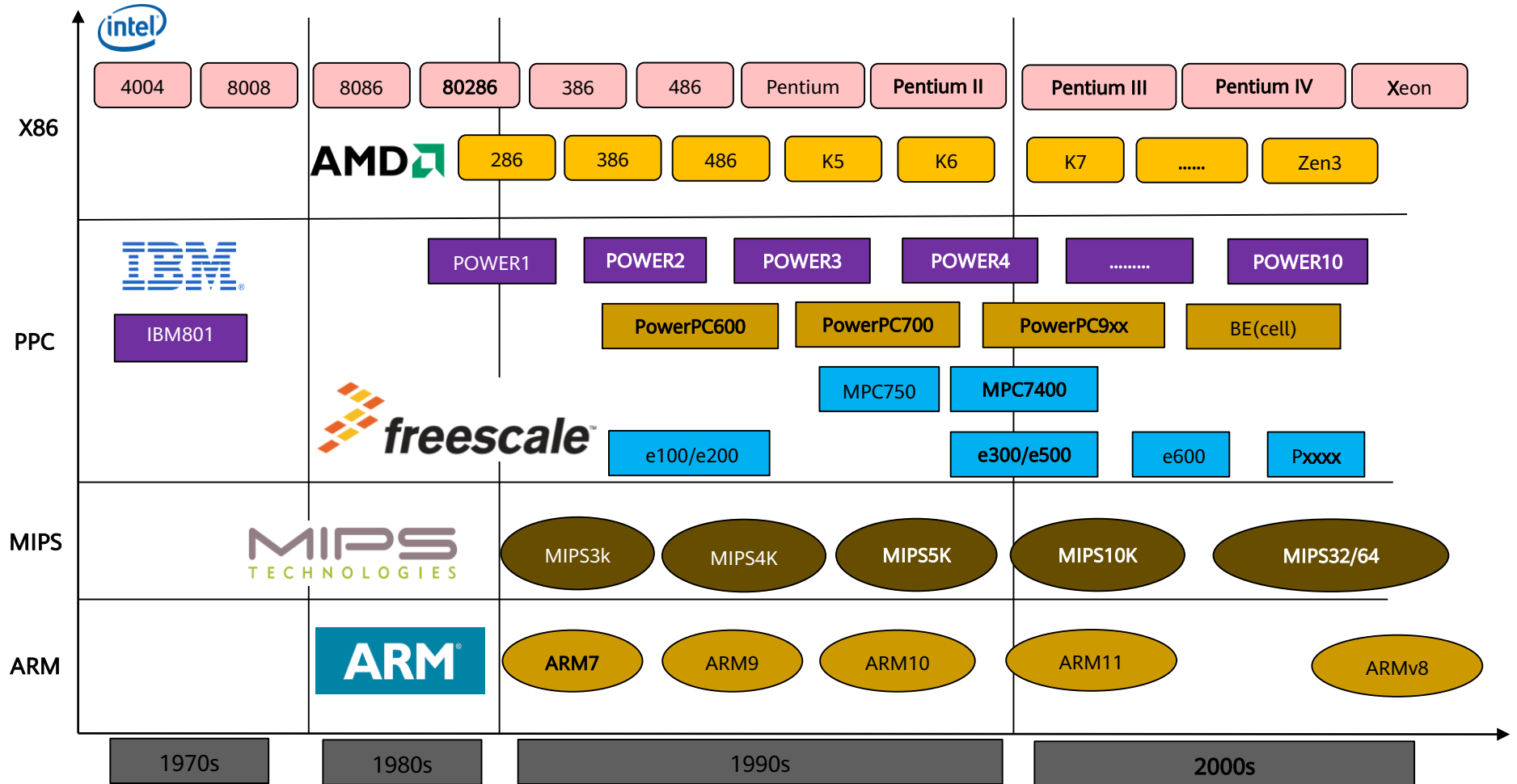
- 精简指令集:保留最基本的,去掉复杂、使用频度不高的指令(选取运算指令、加载、存储指令和转移指令作主指令集),以减小CPI:
$$\text{CPUtime} = \text{IC} * \text{CPI} * \text{CC}$$
 - IC—程序中指令数CPI—每条指令执行所用周期数CC—时钟周期时间
- •复杂指令可通过对简单基本的指令组合而成
- •每条指令的长度都是相同的,大部分指令可以在一个机器周期里完成
- •采用多级指令流水线结构,处理器在同一时间内可执行多条指令
- •采用加载(Load)、存储(Store)结构,统一存储器访问方式,只允许Load和Store指令执行存储器操作,其余指令均对寄存器操作。
- •大大增加通用寄存器的数量,ALU只与寄存器文件直接连接。
- •采用高速缓存(cache)结构,为保证指令不间断地传送给CPU运算器,CPU设置了一定大小的Cache以扩展存储器的带宽,满足CPU频繁取指需求。一般有两个独立Cache,分别存放“指令+数据”

比较内容	CISC	RISC
指令系统	复杂，庞大	简单，精简
指令数目	一般大于200	一般小于100
指令格式	一般大于4	一般小于4
寻址方式	一般大于4	一般小于4
指令字长	不固定	等长
可访存指令	不加限制	只有LOAD/STORE指令
各种指令使用频率	相差很大	相差不大
各种指令执行时间	相差很大	绝大多数在一个周期内完成
优化编译实现	很难	较容易
程序源代码长度	较短	较长
控制器实现方式	绝大多数为微程序控制	绝大多数为硬布线控制
软件系统开发时间	较短	较长

处理器发展趋势



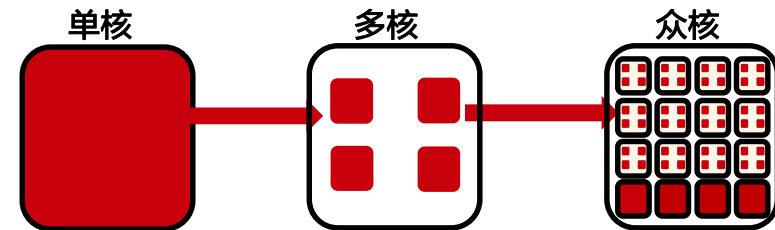
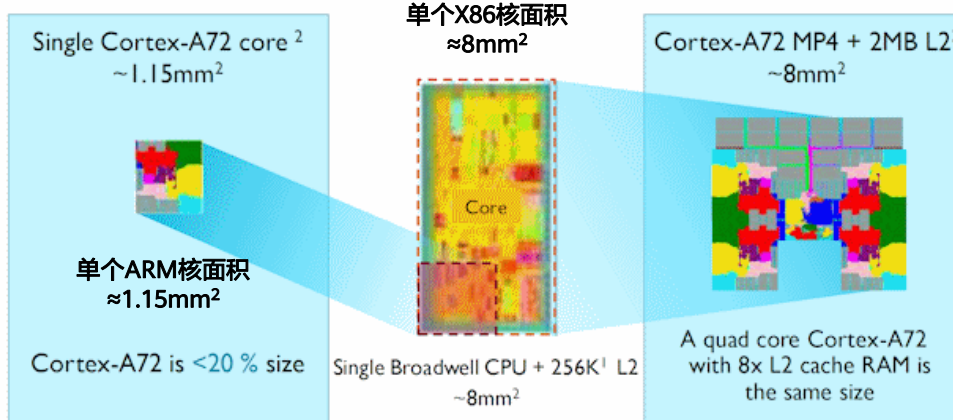
主流CPU发展路径



ARM提供更多计算核心

- 工艺、主频遇到瓶颈后，开始通过增加核数的方式来提升性能；
- 芯片的物理尺寸有限制，不能无限制的增加；
- ARM的众核横向扩展空间优势明显。

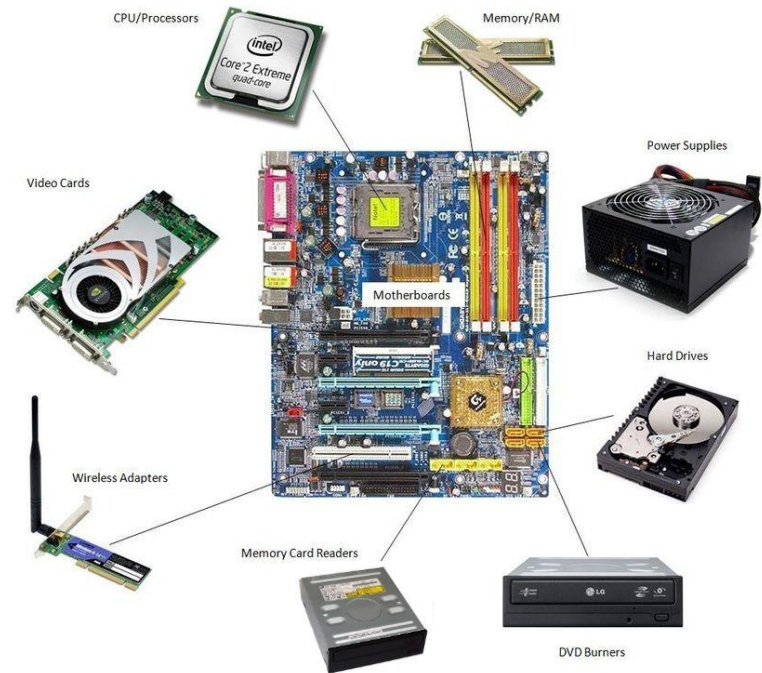
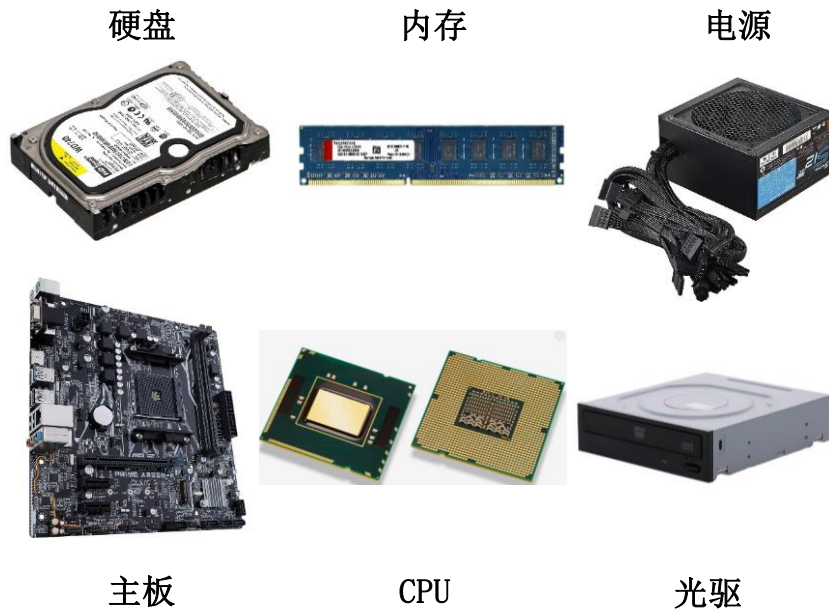
Cortex-A72: Ideal for dense compute environments



ARM服务器级别处理器一览

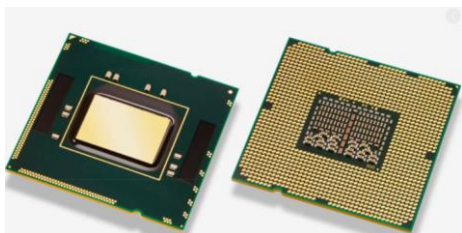
海思 	Hi1612	32C, 2.1GHz 16nm	Hi1616	32C, 2.4GHz 16nm	Hi1620	64C, 3.0GHz 7nm
CAVIUM	Thunder-X	48C, 2.5GHz z 28nm	Thunder-X2	32-54C, 3.0GHz 14nm		
高通			2400	48Cores, 2.2-2.6GHz 14nm		
AMPERE					X-Gene3	32C, 3.3GHz 16nm
飞腾	FT1500	16 Cores, 1.6GHz 28nm	FT2000+	64Cores, 2.3GHz 16nm		
						横轴代表性能

计算机基础硬件 (1)



组成计算机的基础硬件都需要与主板（Motherboard）连接

计算机基础硬件 (2)



CPU

中央处理器是一块集成电路，是一台计算机的运算核心和控制核心。



内存

内存是介于CPU 和外部存储之间，是CPU 高速运算时存放程序指令、数据和中间结果的临时场所，它的物理实质就是一组具备数据输入输出和数据存储功能的高速集成电路。



硬盘

常见的硬盘种类（按接口类型）：

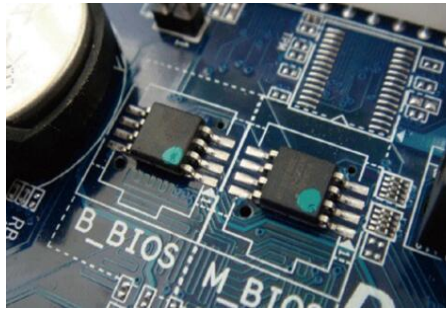
- SATA (Serial ATA)硬盘
- SAS (Serial Attached SCSI)硬盘
- M.2 硬盘（介质为SSD）
- NVMe 硬盘（介质为SSD）

计算机基础硬件 (3)



PCIe卡

PCI Express接口, 简称PCIe, 是计算机总线的一种, 它沿用现有的PCI编程概念及通信标准, 基于更快的串行通信系统。主板上的PCIe插槽通过插接不同的扩展卡如GPU卡、RAID卡、网卡等获得额外的扩展。



BIOS

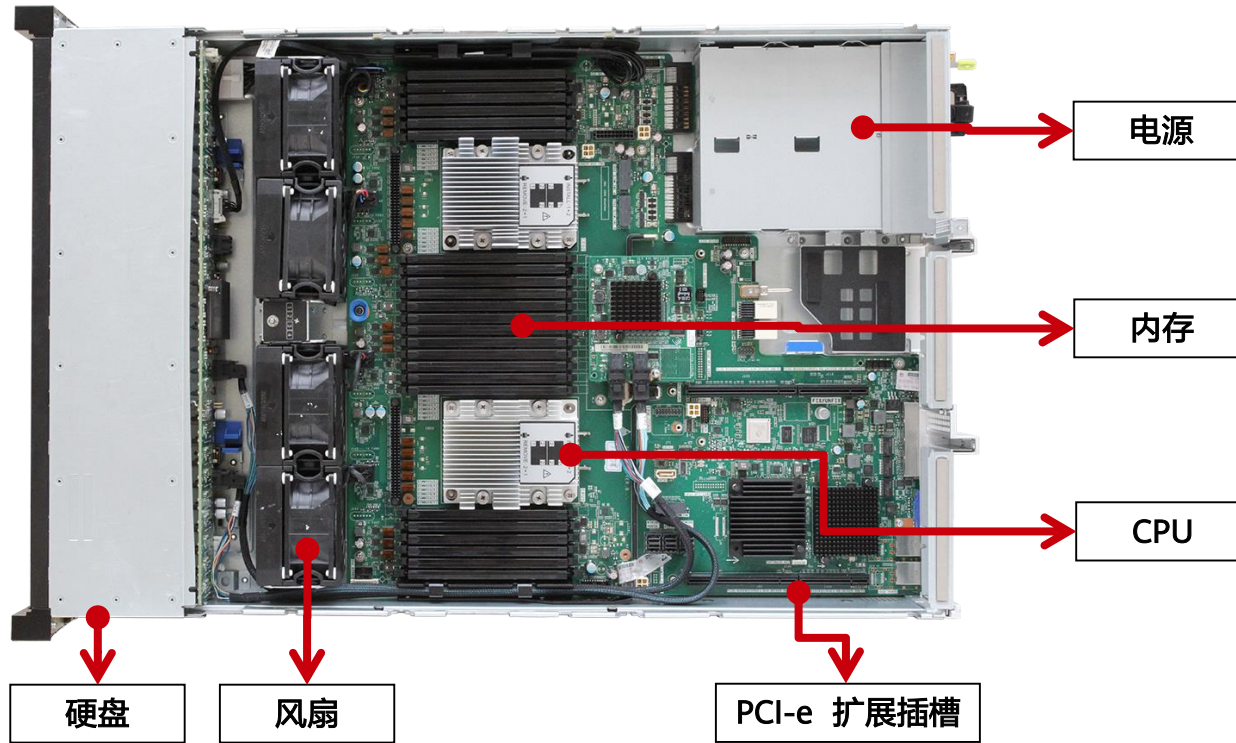
BIOS是Basic Input/Output System的缩写, 基本输入输出系统, 亦称为ROM BIOS、System BIOS, 是在通电引导阶段运行硬件初始化, 以及为操作系统和程序提供运行时服务的组件。



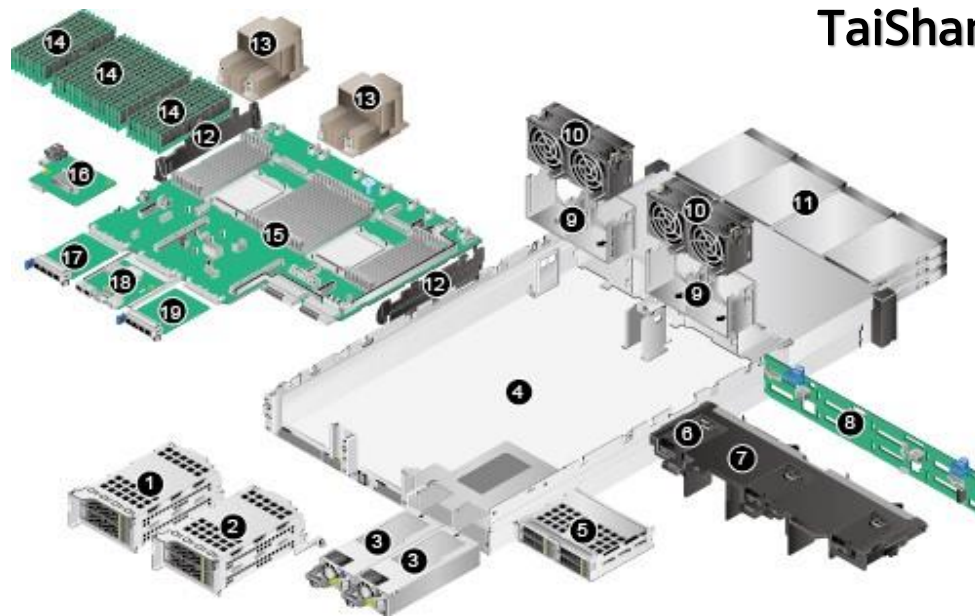
BMC

BMC是基板管理控制器 Baseboard Management Controller。它可以在机器未开机的状态下, 对服务器进行固件升级, 查看设备状态等操作。

服务器基础硬件



TaiShan服务器物理结构

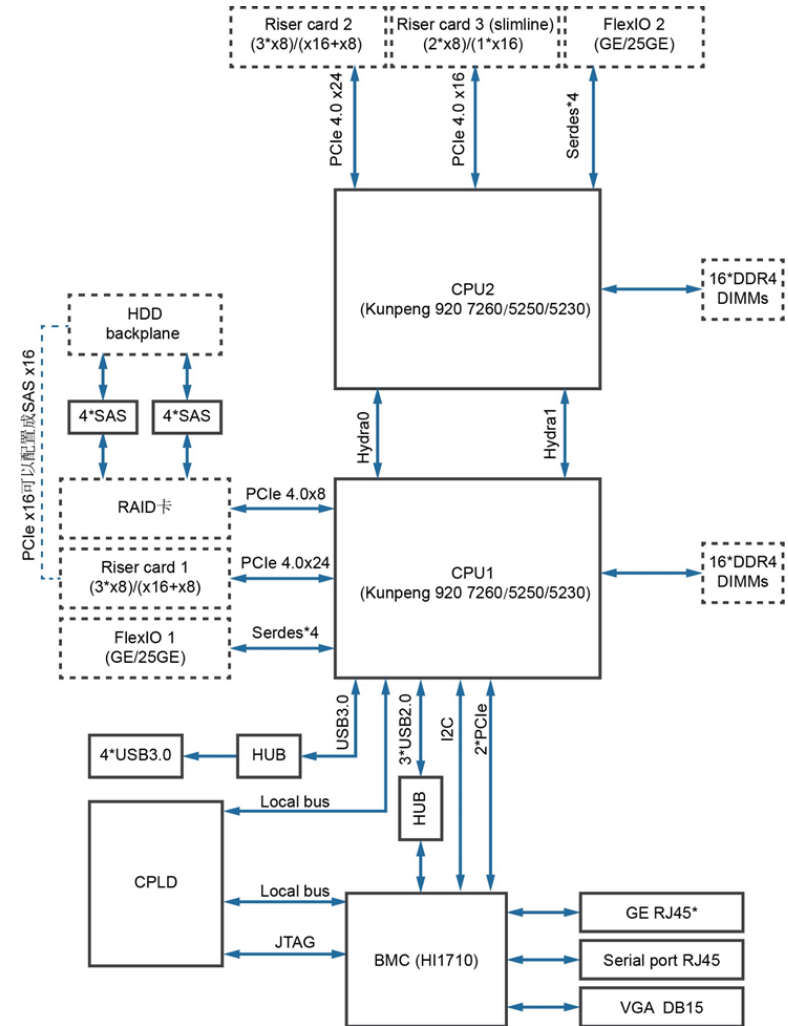


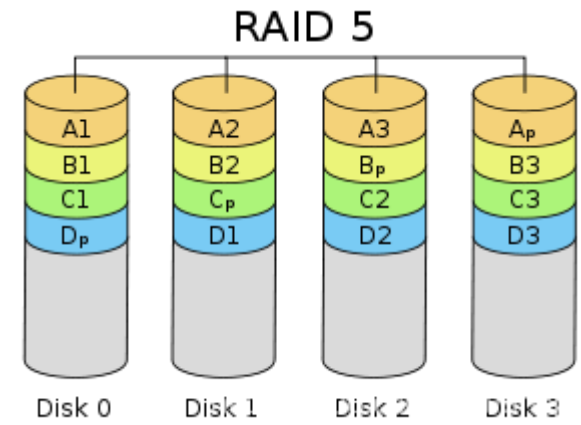
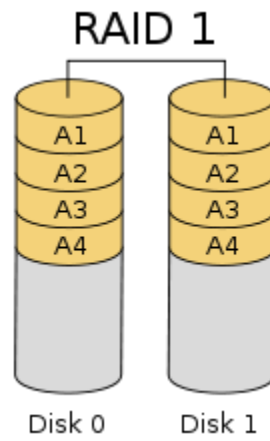
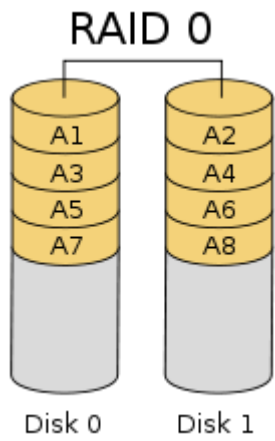
TaiShan 200服务器 (型号2280)

- | | | | |
|----|-------------------|----|--------|
| 1 | I/O模组1 | 2 | I/O模组2 |
| 3 | 电源模块 | 4 | 机箱 |
| 5 | I/O模组3 | 6 | 超级电容支架 |
| 7 | 导风罩 | 8 | 前置硬盘背板 |
| 9 | 风扇支架 | 10 | 风扇模块 |
| 11 | 前置硬盘 | 12 | 理线架 |
| 13 | 散热器 | 14 | DIMM |
| 15 | 主板 | 16 | RAID扣卡 |
| 17 | 灵活I/O卡1 (CPU 1) | 18 | iBMC插卡 |
| 19 | 灵活I/O卡2 (CPU 2) | - | - |

TaiShan服务器逻辑结构

- 支持两路自研鲲鹏920 7260、5250、5240或5230处理器，每个处理器支持16个DDR4 DIMM；
- 以太网灵活插卡可支持2种插卡，包括4*GE和4*25GE，通过CPU本身自带高速Serdes接口连接通信介质；
- RAID卡通过PCI-e总线跟CPU1连接，RAID卡引出的SAS信号线缆跟硬盘背板连接，通过不同的硬盘背板可支持多种本地存储规格；
- BMC使用自研管理芯片Hi1710，可外出VGA、管理网口、调试串口等管理接口。





四大CPU体系结构

- ARM主要应用在移动设备，正在进军server市场
- X86主要AMD的锐龙和Intel的酷睿，其中Intel碾压AMD
- PowerPc主要是IBM自身的server在使用，早年Apple也使用PPC作为Apple的PC的CPU
- MIPS在32位和64位嵌入式领域中历史悠久，获得了不少的成功，可目前Android的采用率在三者中最低



x86系列/Atom处理器

- $xx86$ 或 $80x86$ 是英代尔Intel首先开发制造的一种微处理器体系结构的泛称。
- $x86$ 架构是重要地可变指令长度的CISC（复杂指令集电脑，Complex Instruction Set Computer）。
- Intel Atom（中文：凌动，开发代号：Silverthorne）是Intel的一个**超低电压处理器系列**。处理器采用45纳米工艺制造，集成4700万个晶体管。L2缓存为512KB，支持SSE3指令集，和VT虚拟化技术（部份型号）。现时，Atom处理器系列有6个型号，全部都是属于Z500系列。它们分别是Z500、Z510、Z520、Z530、Z540和Z550。最低端的Z500内核频率是800MHz，FSB则是400MHz。而最高速的Z550，内核频率则有2.0GHz，FSB则是533MHz。从Z520开始，所有的处理器都支持超线程技术，但只增加了不到10%的耗电。双内核版本为N系列，依然采用945GC芯片组。双内核版本仍会支持超线程技术，所以系统会显示出有4个逻辑处理器。这个版本的两个内核并非采用本地设计，只是简单的将两个单内核封装起来。

ARM系列

- ARM架构，过去称作进阶精简指令集机器（Advanced RISC Machine，更早称作：Acorn RISC Machine），是一个32位精简指令集（RISC）处理器架构，其广泛地使用在许多嵌入式系统设计。由于节能的特点，ARM处理器非常适用于行动通讯领域，符合其主要设计目标为低耗电的特性。
- ARM家族占了所有32位嵌入式处理器75%的比例，使它成为占全世界最多数的32位架构之一。ARM处理器可以在很多消费性电子产品上看到，从便携式装置（PDA、移动电话、多媒体播放器、掌上型电子游戏，和计算机）到电脑外设（硬盘、桌上型路由器）甚至在导弹的弹载计算机等军用设施中都有他的存在。
- 在此还有一些基于ARM设计的派生产品，重要产品还包括Marvell的XScale架构和德州仪器的OMAP系列。优势：价格低；能耗低；



MIPS系列

- MIPS是世界上很流行的一种RISC处理器。是“无内部互锁流水级的微处理器” (Microprocessor without interlocked piped stages)，其机制是尽量利用软件办法避免流水线中的数据相关问题。
- 最早80年代初斯坦福Hennessy教授研究小组研制出来的。
- MIPS公司的R系列就是在此基础上开发的RISC工业产品的微处理器。这些系列产品为很多计算机公司采用构成各种工作站和计算机系统。MIPS技术公司是美国著名的芯片设计公司， RISC来设计芯片。
- RISC具有设计更简单、设计周期更短等优点，并可以应用更多先进的技术，开发更快的下一代处理器。MIPS是出现最早的商业RISC架构芯片之一，新的架构集成了所有原来MIPS指令集，并增加了许多更强大的功能。MIPS自己只进行CPU的设计。

PowerPC系列

- 一种精简指令集（RISC）的CPU，其基本的设计源自IBMPowerPC 601微处理器POWER（Performance Optimized With Enhanced RISC；2007年8月号译为“增强RISC性能优化”）架构。
- 二十世纪九十年代，IBM(国际商用机器公司)、Apple（苹果公司）和Motorola（摩托罗拉）公司开发PowerPC芯片成功，并制造出基于PowerPC的多处理器计算机。PowerPC架构的特点是可伸缩性好、方便灵活。PowerPC 处理器有广泛的实现范围，包括从诸如 Power4 那样的高端服务器 CPU 到嵌入式 CPU 市场（任天堂 Gamecube 使用了 PowerPC）。PowerPC 处理器有非常强的嵌入式表现，因为它具有优异的性能、较低的能量损耗以及较低的散热量。

二、ARM系统结构概览

- 1.ARM简介
- 2.ARM7TDMI
- 3.ARM的模块、内核和功能框图

2.1 ARM简介

■ ARM公司简介

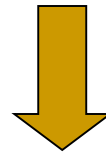
ARM是Advanced RISC Machines的缩写，它是一家微处理器行业的知名企业，该企业设计了大量高性能、廉价、耗能低的RISC（精简指令集）处理器。

ARM 公司的特点是只设计芯片，而不生产。它将技术授权给世界上许多著名的半导体、软件和OEM厂商，并提供服务。

2.1 ARM简介

■ ARM公司简介

ARM[®]



将技术授权给
其它芯片厂商



形成各具特色
的ARM芯片



ATAP Partners

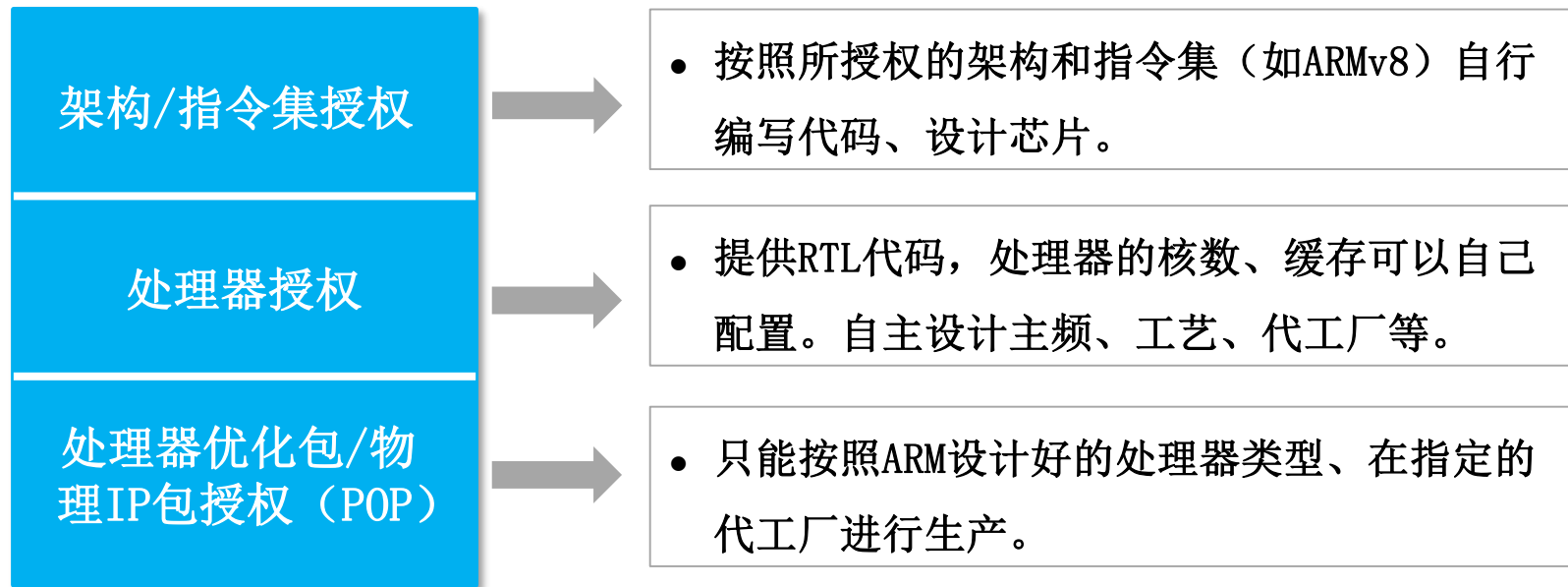


Tools Partners



ARM公司授权体系

- ARM目前在全球拥有大约1000个授权合作商、320家伙伴，但是购买架构授权的厂家不超过20家，中国有华为、飞腾获得了架构授权。



ARM公司的发展历程

- ARM=Advanced RISC Machines
 - – 1985年4月26日，第一个ARM原型在英国剑桥的Acorn计算机有限公司诞生，由美国加州San Jose VLSI技术公司制造。
 - – 20世纪80年代后期，ARM很快开发成Acorn的台式机产品，形成英国的计算机教育基础。
- •1990年成立ARM公司
- •20世纪90年代，ARM32位嵌入式处理器占据了低功耗、低成本和高性能的嵌入式系统应用领域的领先地位。
 - – 1999年因移动电话火爆市场，其32位RISC处理器占市场份额超过了50%
 - – 2001年初，ARM公司的32位RISC处理器市场占有率超过了75%。

2.1 ARM简介

■ ARM体系结构

为了使ARM能够更好地满足嵌入式应用的需要，ARM体系结构还有以下特点：

- 每条数据处理指令可同时包含算术逻辑单元（ALU）的运算和移位处理，实现ALU和移位器的最大利用；
- 使用地址自增和自减的寻址方式优化程序循环；
- 装载/保存指令对数据的批量传输，实现最大数据吞吐量；
- 大多数指令的条件执行，实现最快速的代码执行。

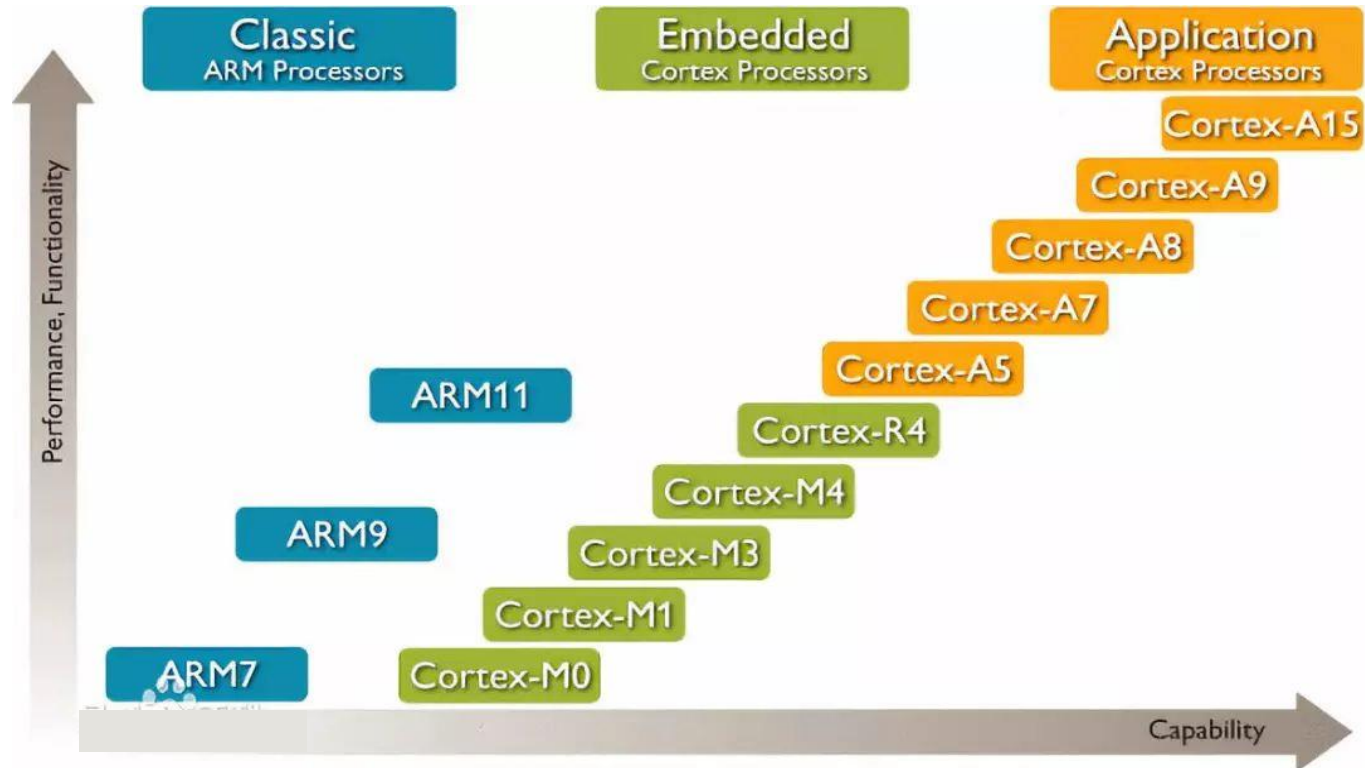
2.1 ARM简介

■ 常用ARM处理器系列

ARM公司开发了很多系列的ARM处理器核，目前最新的系列是Cortex，而ARM6核以及更早的系列已经很罕见了。当前应用比较多的ARM处理器核系列有：

架构	处理器家族
ARMv1	ARM1
ARMv2	ARM2、ARM3
ARMv3	ARM6、ARM7
ARMv4	Strong ARM、ARM7TDMI、ARM9TDMI
ARMv5	ARM7EJ、ARM9E、ARM10E、Xscale
ARMv6	ARM11、ARM Cortex-M
ARMv7	ARM Cortex-A、ARM Cortex-M、ARM Cortex-R
ARMv8	Cortex-A50,Cortex-A57,Cortex-A72

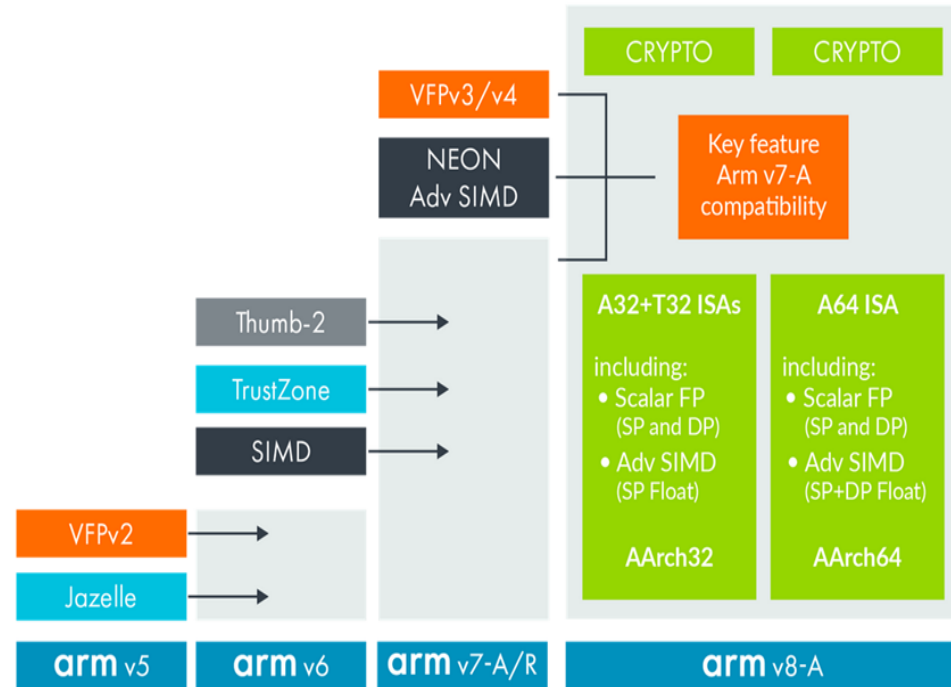
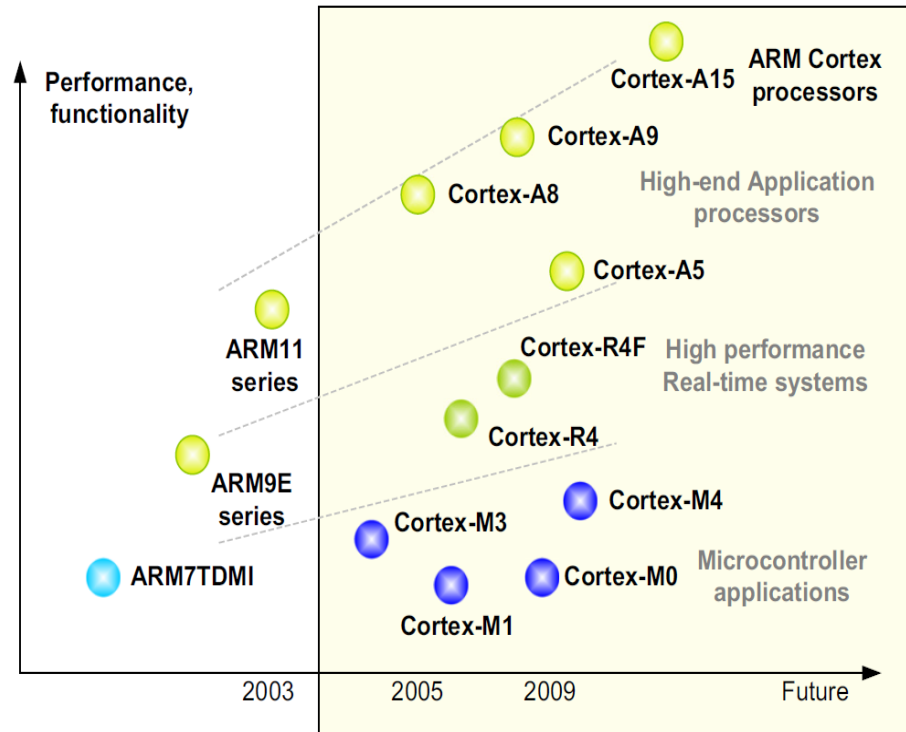
ARM架构发展史 (2)



ARM架构发展史 (3)

- 从ARMv7开始，CPU命名为Cortex，并划分为A、R、M三大系列，分别为不同的市场提供服务
 - A (Application)系列：应用型处理器，面向具有复杂软件操作系统的面向用户的应用，为手机、平板、AP等终端设备提供全方位的解决方案；
 - R (Real-Time)系列：实时高性能处理器，为要求可靠性、高可用性、容错功能、可维护性和实时响应的嵌入式系统提供高性能计算解决方案；
 - M (Microcontroller)系列：高效能、易于使用的处理器，主要用于通用低端，工业，消费电子领域微控制器。

ARM架构发展史 (4)



V1和V2版架构

- • **V1版架构**只在原型机**ARM1**出现过,其基本性能:
 - – 基本的数据处理指令(无乘法)
 - – 字节、半字和字的LOAD/STORE指令
 - – 转移指令,包括子程序调用及链接指令
 - – 软件中断指令
 - – 寻址空间: 64M字节(26位寻址空间)
- • **V2版对V1版进行了扩展,如ARM2与ARM3(V2a版)**
 - 架构,增加了以下功能:
 - – 乘法和乘加指令
 - – 支持协处理器操作指令
 - – 快速中断模式
 - – SWP/SWPB的最基本存储器与寄存器交换指令

V3版架构

- 把寻址空间增至32位(4G字节)
- 增加了当前程序状态寄存器CPSR(Current Program Status Register)和程序状态保存寄存器 SPSR(Saved Program Status Register)以便于异常(Exception)的处理。
- 增加了中止(Abort)和未定义二种处理器模式。
- 指令集变化如下：
 - – 增加了MRS/MSR指令,以访问新增的CPSR/SPSR寄存器
 - – 增加了从异常处理返回的指令功能
 - – T —— Thumb状态: 16位指令执行时可以无开销地扩展成32位指令格式
 - – M ——长乘法支持: ($32*32 \Rightarrow 64$ 或者 $32*32+64 \Rightarrow 64$)
- ARM6就采用该版架构。

V4版架构

- 对V3版架构进行了进一步扩充,有的还引进了**16位**的Thumb指令集,使ARM使用更加灵活。
- D ——对调试的支持 (Debug) ,JTAG接口, 片上调试
- I ——嵌入的ICE (In Circuit Emulation) ,允许对电路内部嵌入的ARM内核进行访问和控制, 片上断点
- 指令集中增加了以下功能:
 - – 符号化和非符号化半字及符号化字节的存/取指令
 - – 增加了**16位Thumb**指令集
 - – 完善了软件中断SWI指令的功能
 - – 处理器系统模式引进特权方式时使用用户寄存器操作
 - – 把一些未使用的指令空间捕获为未定义指令
- ARM7、部分ARM9和StrongARM都采用该版架构

V5版架构

- • 在V4版基础上增加了一些新的指令, 主要提升了ARM和Thumb指令的交互工作能力。
- • E ——DSP指令支持。
- • J ——Java指令支持。
- • 新增指令有:
 - – 带有链接和交换的转移BLX指令
 - – 计数前导零CLZ指令
 - – BRK中断指令, 软件断点
 - – 增加了信号处理指令(V5TE版)
 - – 为协处理器增加更多可选择的指令
- • ARM10和XScale都采用该版架构。

V6版架构

- 2001年发布的。
- • 在V5版基本上增加了增加了媒体指令(SIMD, FFT, MPEG4),可以使音频/视频处理性能提高4倍。
- • 提升了性能与功耗比, 适合使用电池供电的便携式设备
- • 多处理器支持(multiprocessing)
- • 改善的异常和中断处理能力
- • 新的内存管理机制
- • 混序数据支持
- • ARM11采用该版架构。

V7版架构

- ARMv7架构是在ARMv6架构的基础上诞生的。
- 采用了**Thumb-2**技术，Thumb-2技术是在ARM的Thumb代码压缩技术的基础上发展起来的，并且保持了对现存ARM解决方案的完整的代码兼容性。Thumb-2技术比纯32位代码少使用 31% 的内存，减小了系统开销。同时能够提供比已有的基于Thumb技术的解决方案高出38%的性能。
- ARMv7架构还采用了**NEON技术**，将DSP和媒体处理能力提高了近4倍，并支持改良的浮点运算，满足下一代3D图形、游戏物理应用以及传统嵌入式控制应用的需求。此外，ARMv7还支持改良的运行环境，以迎合不断增加的JIT(Just In Time)和DAC(Dynamic Adaptive Compilation)技术的使用。
- ARMv7架构对于早期的ARM处理器软件也提供很好的兼容性。ARMv7架构定义了三大分工明确的系列：“A”系列面向尖端的基于虚拟内存的操作系统和用户应用；“R”系列针对实时系统；“M”系列对微控制器和低成本应用提供优化。

V8版架构

- **ARMv8-A 将 64 位架构支持引入 ARM 架构中，包括：**
 - 64 位通用寄存器、SP（堆栈指针）和 PC（程序计数器）
 - 64 位数据处理和扩展的虚拟寻址
 - 两种主要执行状态：
 - AArch64 - 64 位执行状态，包括该状态的异常模型、内存模型、程序员模型和指令集支持
 - AArch32 — 32 位执行状态，包括该状态的异常模型、内存模型、程序员模型和指令集支持
- **三个主要指令集：**
 - A32（或 ARM）：32 位固定长度指令集，通过不同架构变体增强部分 32 位架构执行环境现在称为 AArch32。
 - T32 (Thumb) 是以 16 位固定长度指令集的形式引入的，随后在引入 Thumb-2 技术时增强为 16 位和 32 位混合长度指令集。部分 32 位架构执行环境现在称为 AArch32。
 - A64：提供与 ARM 和 Thumb 指令集类似功能的 32 位固定长度指令集。随 ARMv8-A 一起引入，它是一种 AArch64 指令集。

	Thumb	DSP	Jazelle	Media	TrustZone	Thumb-2	
v4							StrongARM
v4T	*						ARM7T, ARM9
v5T	*						ARM10T, XScale
v5TE	*	*					ARM9E, ARM10E
v5TEJ	*	*	*				ARM7EJ, ARM9EJ, ARM10EJ
v6	*	*	*	*			ARM1136J(F)-S
v6Z	*	*	*	*	*		
v6T2	*	*	*	*		*	ARM1156T2(F)-S

2.1 ARM简介

■ ARM Cortex系列简介

基于ARMv7版本的ARM Cortex系列产品由A、R、M三个系列组成，具体分类延续了一直以来ARM面向具体应用设计CPU的思路。



2.1 ARM简介

■ Cortex™-M3处理器简介

该处理器是首款基于ARMv7-M架构的处理器，采用了纯Thumb2指令的执行方式，具有极高的运算能力和中断相应能力。

Cortex-M3主要应用于汽车车身系统，工业控制系统和无线网络等对功耗和成本敏感的嵌入式应用领域。目前最便宜的基于该内核的ARM单片机售价为1美元。

2.1 ARM简介

■ Cortex™-R4处理器简介

该处理器是首款基于ARMv7架构的高级嵌入式处理器，其主要目标为产量巨大的高级嵌入式应用系统，如硬盘，喷墨式打印机，以及汽车安全系统等等。

● Cortex™-R4F处理器简介

该处理器在Cortex™-R4处理器的基础上加入了代码错误校正(ECC)技术，浮点运算单元(FPU)以及DMA综合配置的能力，增强了处理器在存储器保护单元、缓存、紧密耦合存储器、DMA访问以及调试方面的能力。

2.1 ARM简介

■ Cortex™-A8处理器简介

该处理器是ARM公司所开发的基于ARMv7架构的首款应用级处理器，其特色是运用了可增加代码密度和加强性能的技术、可支持多媒体以及信号处理能力的NEON™技术、以及能够支持Java和其他文字代码语言的提前和即时编译的Jazelle®RTC技术。

众多先进的技术使其适用于家电以及电子行业等各种高端的应用领域。

2.1 ARM简介

■ ARM7系列简介

该系列包括ARM7TDMI、ARM7TDMI-S、带有高速缓存处理器宏单元的ARM720T和扩充了Jazelle的ARM7EJ-S。该系列处理器提供Thumb 16位压缩指令集和EmbeddedICE软件调试方式，适用于更大规模的SoC设计中。

ARM7系列广泛应用于多媒体和嵌入式设备，包括Internet设备、网络和调制解调器设备，以及移动电话、PDA等无线设备。

2.1 ARM简介

■ ARM9系列简介

该系列包括ARM9TDMI、ARM920T和带有高速缓存处理器宏单元的ARM940T。除了兼容ARM7系列，而且能够更加灵活的设计。

ARM9系列主要应用于引擎管理、仪器仪表、安全系统和机顶盒等领域。

2.1 ARM简介

■ ARM9E系列简介

该系列为含有DSP指令集的综合处理器，包括ARM926EJ-S、带有高速缓存处理器宏单元的ARM966E-S/ARM946E-S。其内核在ARM7处理器内核的基础上使用了Jazelle增强技术，该技术支持一种新的Java操作状态，允许在硬件中执行Java字节码。

ARM9E系列主要应用于下一代无线设备、数字消费品、成像设备、工业控制、存储设备和网络设备等领域。

2.1 ARM简介

■ ARM10E系列简介

该系列包括ARM1020E和ARM1020E处理器核，其核心在于使用向量浮点（VFP）单元VFP10提供高性能的浮点解决方案，从而极大提高了处理器的整型和浮点运算性能。

可以用于视频游戏机和高性能打印机等场合。

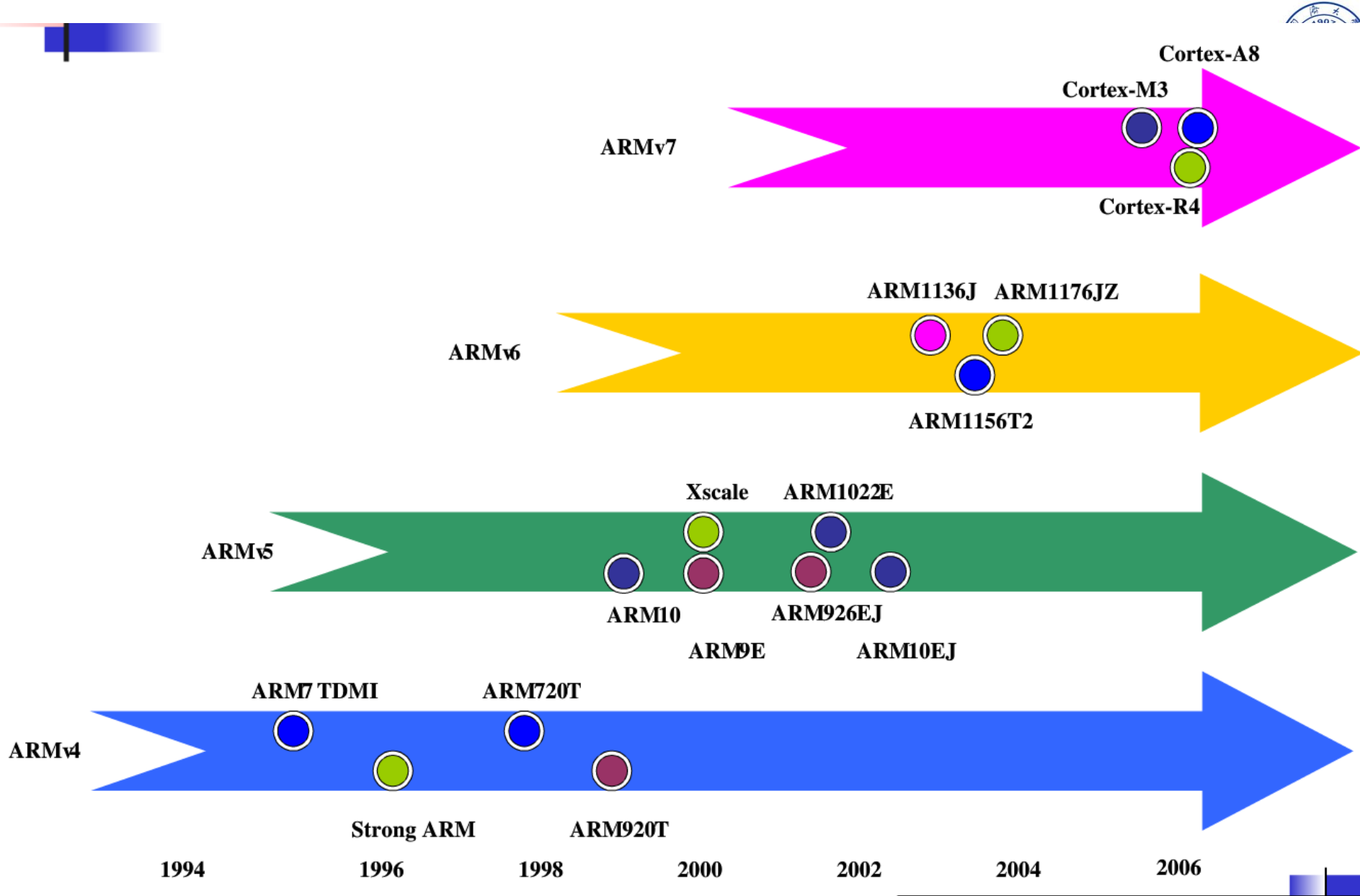


2.1 ARM简介

■ Xscale简介

Intel Xscale微控制器则提供全性能、高性价比、低功耗的解决方案，支持16位Thumb指令并集成数字信号处理（DSP）指令。

主要应用于手提式通讯和消费电子类设备。



ARMv8指令集发布于2011年11月，它在ARM历史上第一次支持了64位指令集，构成了苹果2013年能首发64位处理器A9的核心基础。

2016年旗舰手机处理器“自主”和“公版”设计一览表

	高通 骁龙820	联发科 Helio X20	海思 Kirin950	三星 Exynos8890	苹果 A9
CPU	自主设计 Kryo架构	ARM公版 A72+A53架构	ARM公版 A72+A53架构	自主设计 公版 Mongoose+A53架构	自主设计 Twister架构
GPU	自主设计 Adreno530	ARM公版 Mali-T880 MP4	ARM公版 Mali-T880 MP4	ARM公版 Mali-T880 MP12	PowerVR GT7600
CCI	自主设计 L3缓存	基于ARM设计 MCSI	ARM公版 ARM CCI-400	基于ARM设计 SCI	自主设计 L3缓存
ISP	自主研发	未知	自主研发	自主研发	自主研发
LTE	自主研发 Cat.12/13 SoC集成	自主研发 Cat.6测试中 SoC集成	自主研发 Cat.6 SoC集成	自主研发 Cat.12/13 SoC集成	非自主·高通基带 Cat.6 外挂基带

@i冰宇宙

- 骁龙616 a53 $A72 > A57 > A17 > A15 > A9 > A53 > A35 > A8 > A7 > A5$

如今我们常见的手机自主/非自主处理器架构都基于arm指令集（除了少数的Intel核心手机为X86指令集）

骁龙 820 四核Kryo 架构基于 ARMv8 指令集

2.2 ARM7TDMI

■ 简介

注意：ARM7TDMI是“基于ARM核心”的ARM4核与其外设如RAM、ROM、补片ARM4核组合在低于5M功耗或稳定的功耗不足，还增加了后缀所对应的功能：

ARM7TDMI-S

ARM7TDMI的可综合 (synthesizable) 版本 (软核)，对应用工程师来说其编程模型与ARM7TDMI一致；

支持EmbeddedICE观察硬件；

支持64位乘法；

支持片上调试；

支持高密度16位的Thumb指令集；

- • **T**: 内含**16**位压缩指令集**Thumb**
- • **D**: 支持片内**Debug**调试,
- • **M**: 采用增强型乘法器(**Multiplier**),
- • **I**: 内含嵌入式**ICE**宏单元
- • **-S**: 可综合的软核**Softcore**
- • **-E**: 具有**DSP**的功能
- • **-J**: **Jazeller**,允许直接执行**Java**字节码

2.2 ARM7TDMI

■ 存储器的字与半字

从偶数地址开始的连续2个字节构成一个**半字**；

以能被4整除的地址开始的连续4个字节构成一个**字**；

ARM指令的长度刚好是一个字，Thumb指令的长度刚好是一个半字。

2.2 ARM7TDMI

■ 存储器的存储方式

如果一个数据是从偶地址开始的连续存储，那么它就是**半字对齐**，否则就是**非半字对齐**；

如果一个数据是以能被4整除的地址开始的连续存储，那么它就是**字对齐**，否则就是**非字对齐**。

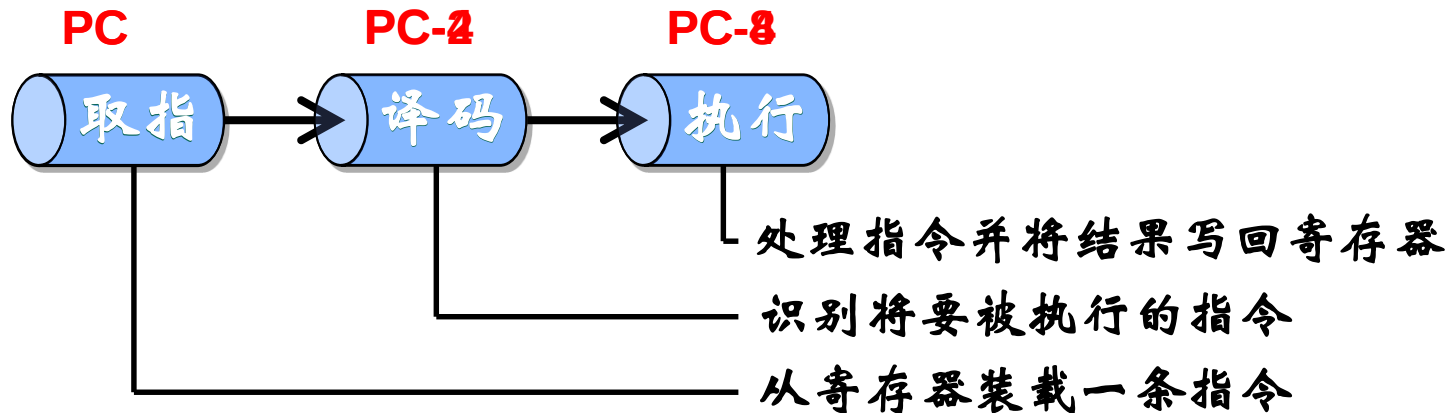
方式	半字对齐	字对齐
地址	 0x4002 0x4004	 0x4004 0x4008
特征	Bit0=0 其他位为任意值	Bit1=0, Bit0=0 其他位为任意值

2.2 ARM7TDMI

■ 三级流水线

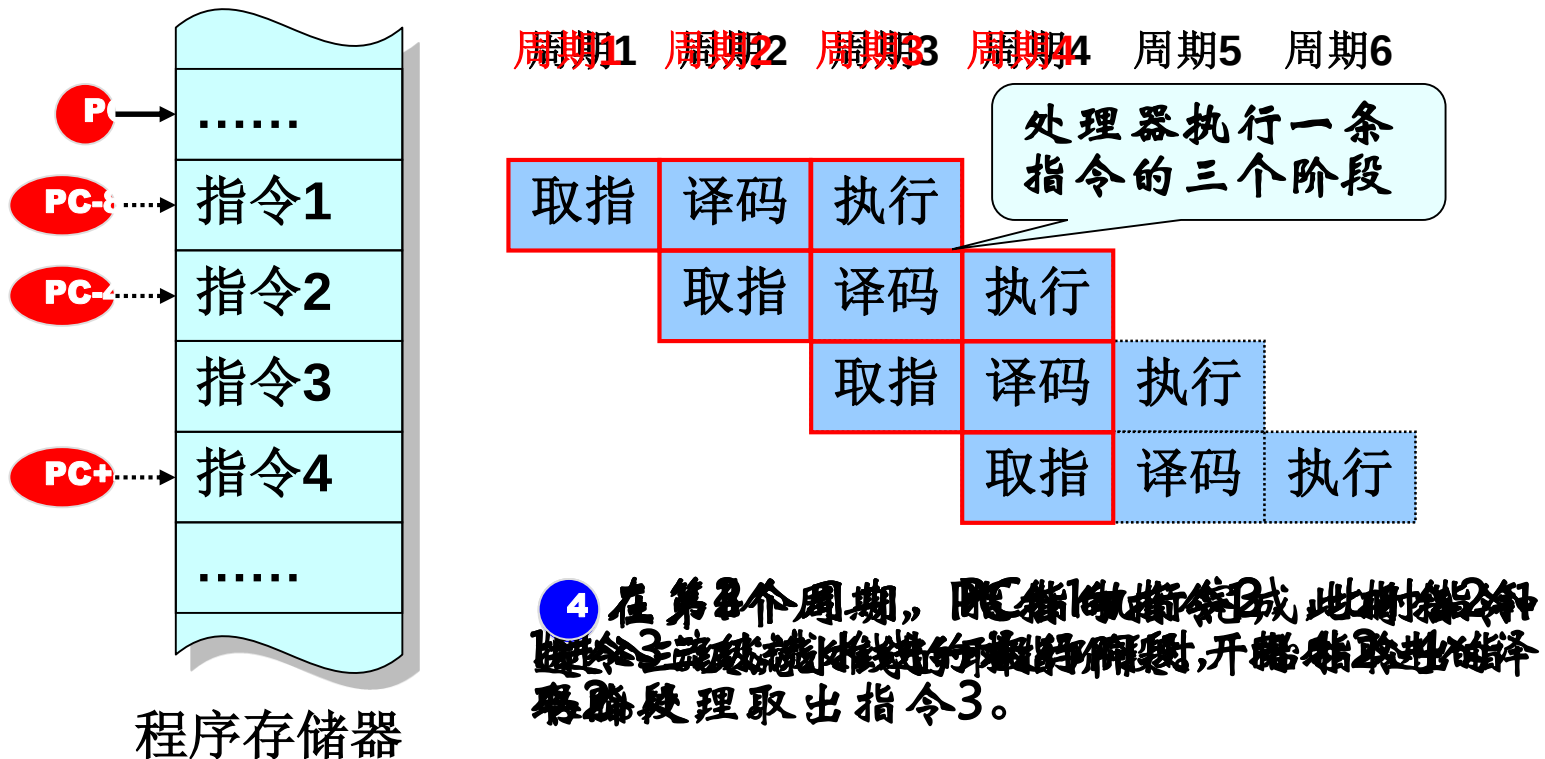
ARM7TDMI处理器采用流水作业方式，将指令的执行过程分为三个阶段：取指、译码和执行。这三个阶段由三个寄存器（PC、PC+4、PC+8）连续操作，能提供0.9MIPS/MHz的指令执行速度。

在ARM7TDMI状态下，流水线的各阶段地址为：
ARM7TDMI的流水线分3级，分别为：



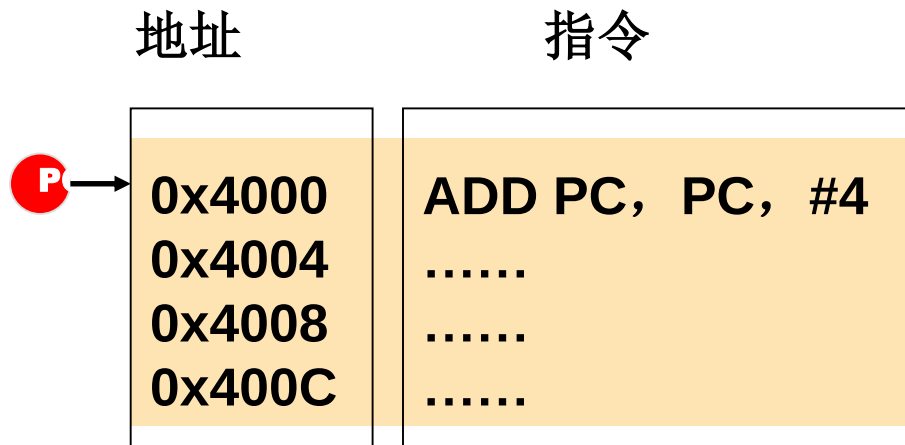
2.2 ARM7TDMI

■ 三级流水线结构的指令执行顺序

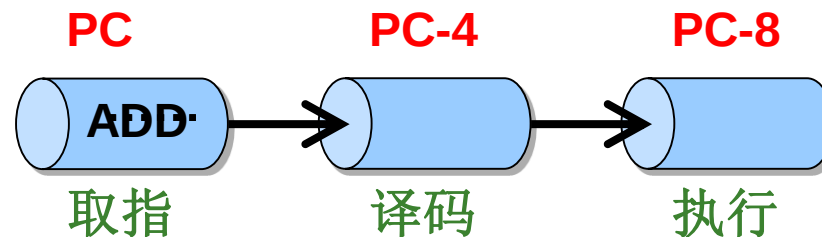


2.2 ARM7TDMI

■ 思考题



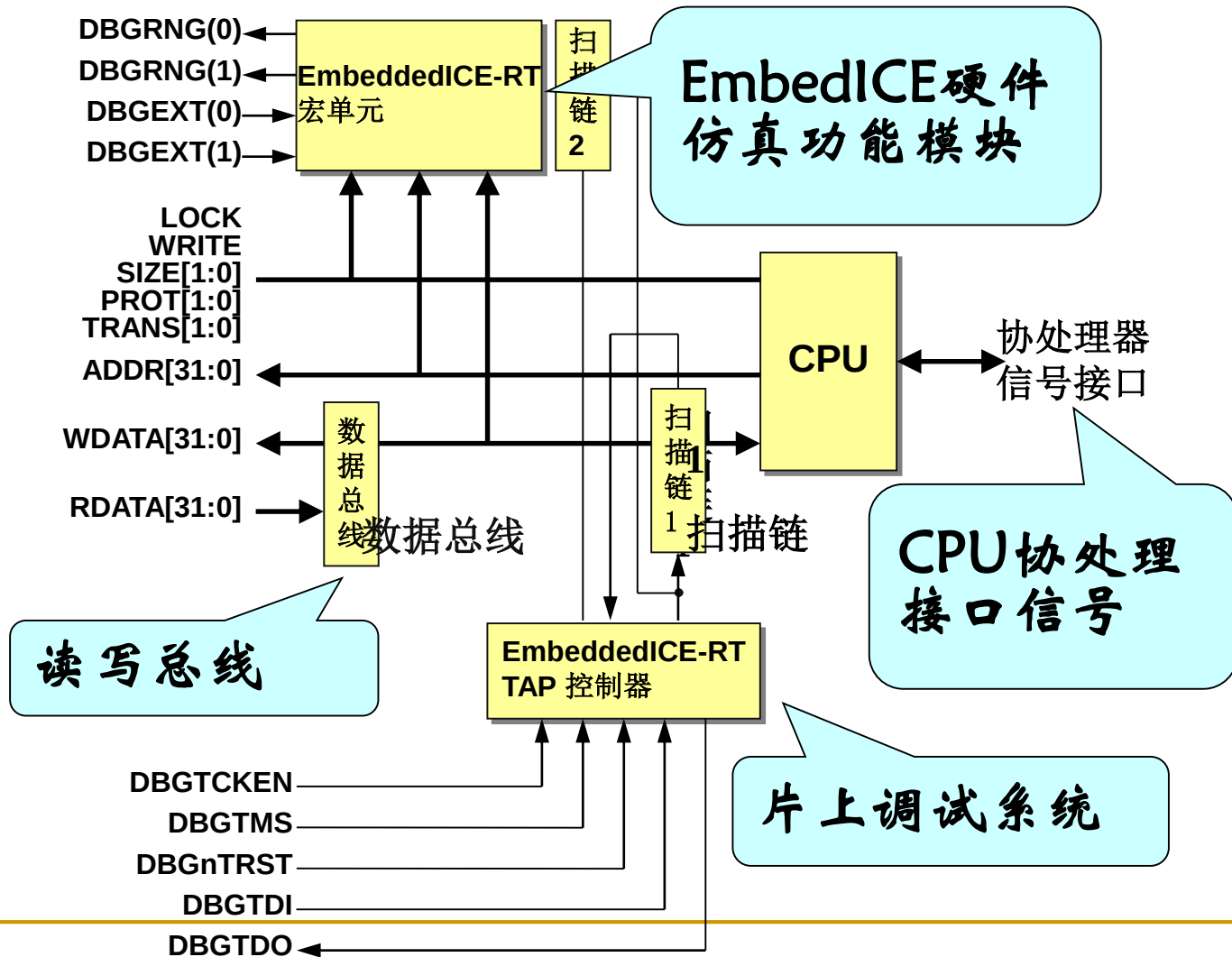
一般来说，人们习惯性约等于“取指”、“译码”、“执行”三个阶段。PC值+4，前一条指令的PC值+8。PC=PC+4，PC=PC+8或4。ARM指令集。每条指令为4字节长。



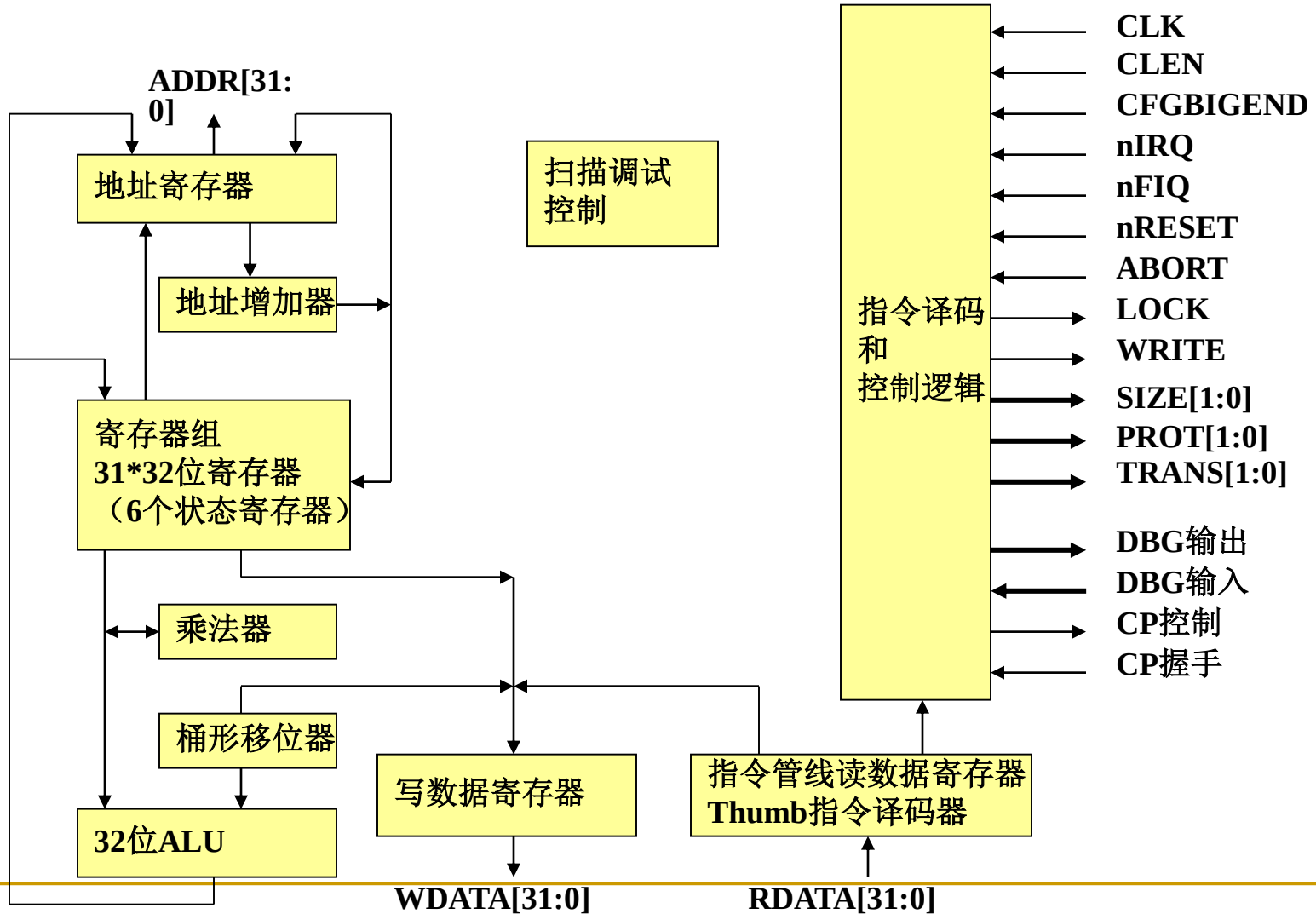
ARM7的3级流水线

ARM7	预取 (Fetch)	译码 (Decode)	执行 (Execute)					
ARM9	预取 (Fetch)	译码 (Decode)	执行 (Execute)	访存 (Memory)	写回 (Write)			
ARM10	预取 (Fetch)	发射 (Issue)	译码 (Decode)	执行 (Execute)	访存 (Memory)	写回 (Write)		
ARM11	预取 (Fetch)	预取 (Fetch)	发射 (Issue)	译码 (Decode)	转换 (Snnny)	执行 (Execute)	访存 (Memory)	写回 (Write)

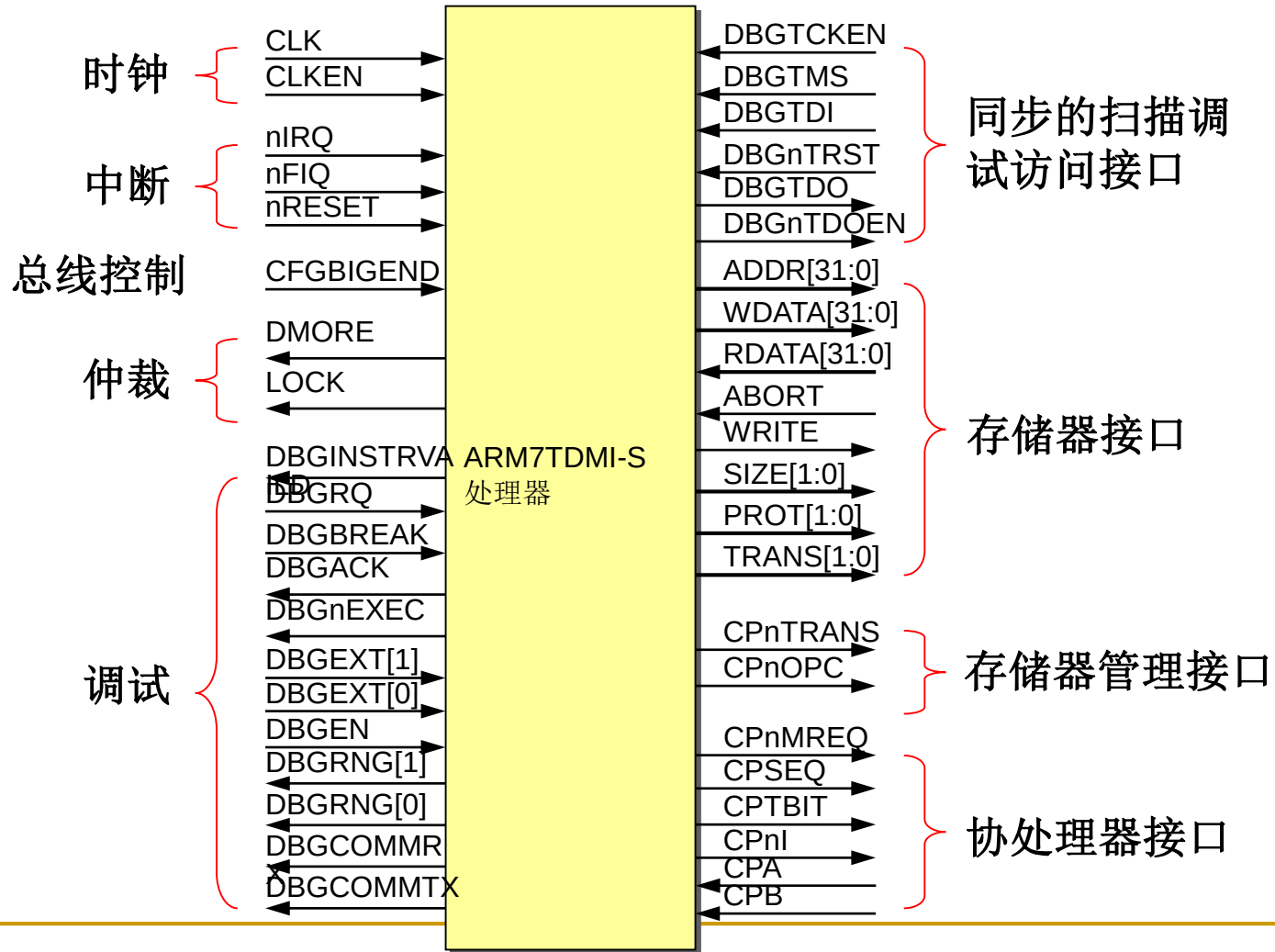
2.3 ARM模块框图



2.3 ARM内核框图



2.3 ARM功能框图



三、ARM编程模型

- 1.ARM处理器状态
 - 2.ARM处理器模式
 - 3.ARM内部寄存器
 - 3.当前程序状态寄存器
 - 4.ARM体系的异常、中断及其向量表
 - 5.ARM体系的存储系统
-

3.1 ARM处理器状态

■ 处理器状态

ARM7TDMI处理器内核包含2套指令系统，分别为ARM指令集和Thumb指令，并且各自对应1种处理器的状态：

- ARM状态：32位，处理器执行字方式的ARM指令，处理器默认为此状态；
- Thumb状态：16位，处理器执行半字方式的Thumb指令。

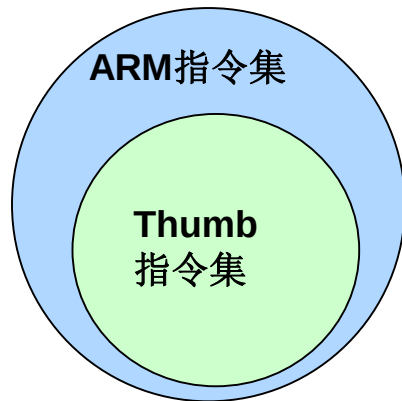
注意：两个状态之间的切换并不影响处理器模式或寄存器内容。

3.1 ARM处理器状态

■ 状态切换的一个例子

使用BX指令将ARM内核的操作状态在ARM状态和Thumb状态之间进行切换。

从ARM状态切换到Thumb状态的程序代码如下：



指令集关系

CODE32		
	LDR	R0, =Lable+1
	BX	R0
CODE32		
Lable	MOV	R1, #10

程序代码

跳转地址标号

地址最低位为1，表示切换到Thumb状态

执行完BX指令，处理器切换到ARM状态，开始执行ARM指令

3.2 ARM处理器模式

- 简介

ARM体系结构支持7种处理器模式，分别为：用户模式、快中断模式、中断模式、管理模式、中止模式、未定义模式和系统模式。这样的好处是可以更好的支持操作系统并提高工作效率。ARM7TDMI完全支持这七种模式。

- 处理器复位之后进入管理模式，操作系统内核通常处于管理模式；
- 当处理器访问存储器失败时，进入数据访问中止模式；
- 当处理器遇到没有定义或不支持的指令时，进入未定义模式；
- 中断模式与快速中断模式分别对ARM处理器2种不同级别的中断作出响应。



3.3 ARM内部寄存器

■ 简介

在ARM处理器内部共有37个用户可访问的寄存器，分别为**31个通用32位寄存器**和**6个状态寄存器**。

ARM处理器共有7种不同的处理器模式，每种模式都有一组相应的寄存器组，最多可以有18个活动的寄存器。

ARM状态各模式下的寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器							
		用户	系统	管理	中止	未定义	中断	快中断	
通用寄存器和程序计数器	R0(a1)	R0							
	R1(a2)	R1							
	R2(a3)	R2							
	R3(a4)	R3							
	R4(v1)	R4							
	所有的37个寄存器，分成两大类： ▪31个通用32位寄存器； ▪6个状态寄存器。		R5						
			R6						
			R7						
			R8					R8_fiq	
			R9					R9_fiq	
			R10					R10_fiq	
			R11					R11_fiq	
	R12(IP)	R12					R12_fiq		
	R13(SP)	R13	R13_scv	R13_abt	R13_und	R13_irq	R13_fiq		
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R13_irq	R14_fiq		
R15(PC)	R15								
状态寄存器	CPSR	CPSR							
	SPSR	—	SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq		

ARM状态各模式下可以访问的寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	R0						
	R1(a2)	R1						
	R2(a3)	R2						
	R3(a4)	R3						
	R4(v1)	R4						
	R5(v2)	R5						
	R6(v3)	R6						
	R7(v4)	R7						
	R8(v5)	R8						R8_fiq
	R9(SB,v6)	R9						R9_fiq
	R10(SL,v7)	R10						R10_fiq
	R11(FP,v8)	R11						R11_fiq
	R12(IP)	R12						R12_fiq
	R13(SP)	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq	
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq	
R15(PC)	R15							
状态寄存器	CPSR	CPSR						
	SPSR	—	SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq	

ARMv7-M 架构的通用寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	寄存器R0~R7为未分组的通用寄存器，			R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)	寄存器R14称为链接寄存器 (LR)，在结构上有两个特殊功能：						
	R5(v2)							
	R6(v3)							
	R7(v4)							
	R8(v5)	寄存器CPSR为当前程序状态寄存器，在异常模式中，另外一个寄存器“程序状态保存寄存器 (SPSR)”可以被访问。每种异常都有自己的SPSR，在进入异常时它保存CPSR的当前值，异常退出时可通过它恢复CPSR。						
	R9(SB,v6)							
	R10(SL,v7)							
	R11(FP,v8)							
	R12(IP)							
	R13(SP)							
	R14(LR)							
	R15(PC)				R15			
状态寄存器	CPSR	CPSR						
	SPSR	—	SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq	

3.3 ARM内部寄存器

■ 在Thumb状态各模式下实际访问的寄存器

Thumb状态寄存器集是ARM状态集的子集，程序员可以直接访问的寄存器为：

- 8个通用寄存器R0~R7；
- 程序计数器（PC）；
- 堆栈指针（SP）；
- 链接寄存器（LR）；
- 当前程序状态寄存器（CPSR）。

在Thumb状态各模式下的寄存器

• 策分組的通用通用寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	堆栈指针SP对应ARM状态的寄存器R13。每个异常模式都有其自身的SP分组版本，链接寄存器LR对应ARM状态寄存器R14。 注意：在发生异常时，处理器自动进入ARM状态。						
	R1(a2)							
	R2(a3)							
	R3(a4)							
	R4(v1)							
	R5(v2)							
	R6(v3)							
	R7(v4,WR)							
	SP							
	LR							
状态寄存器	PC							
	CPSR							

Thumb寄存器在ARM状态寄存器上的映射

Thumb状态

ARM状态

R0	→	R0
R1	→	R1
R2	→	R2
R3	→	R3
R4	→	R4
R5	→	R5
R6	→	R6
R7	→	R7
		R8
		R9
		R10
		R11
		R12
堆栈指针(SP)	→	堆栈指针(R13)
连接寄存器(LR)	→	连接寄存器(R14)
程序计数器(PC)	→	程序计数器(R15)
CPSR	→	CPSR
		SPSR

① Thumb状态R0~R7与ARM状态R0~R7相同；

② **低端寄存器** Thumb状态SP映射到ARM状态R13；

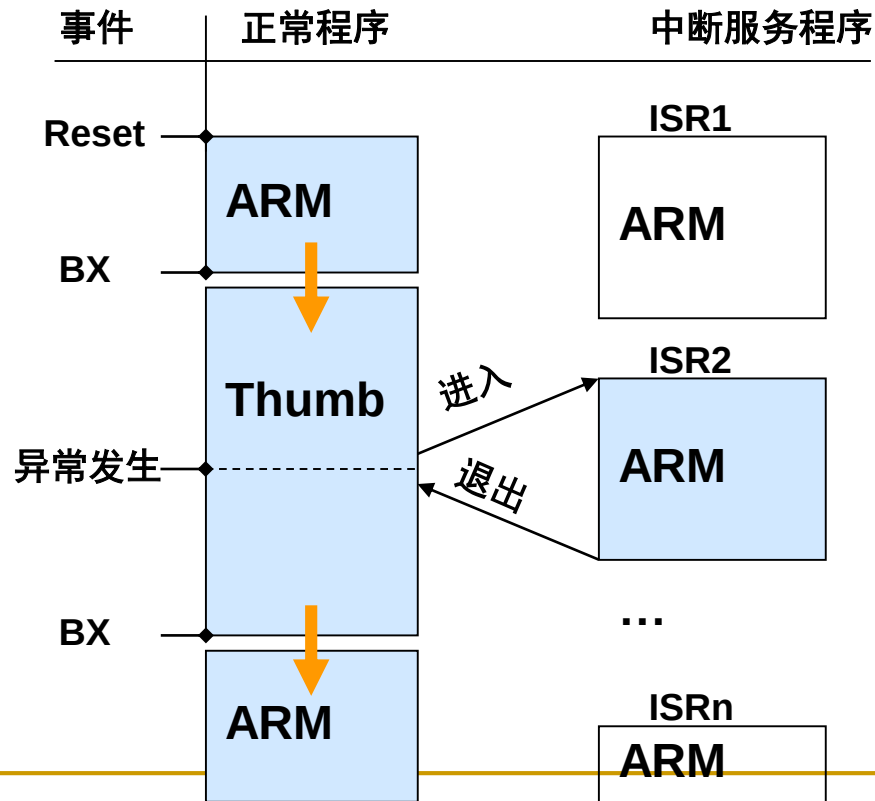
③ Thumb状态LR映射到ARM状态R14；

④ **高端寄存器** Thumb状态PC映射到ARM状态R15(PC)；

⑤ 在Thumb状态中，高端寄存器的访问是受到限制的，只有MOV、CMP和ADD指令可以对其访问，可以用于数据的快速暂存。

■ 状态切换过程

6 程序在正常运行的过程中，复位事件产生，导致程序从ARM状态切换到Thumb状态。



3.4 当前程序状态寄存器

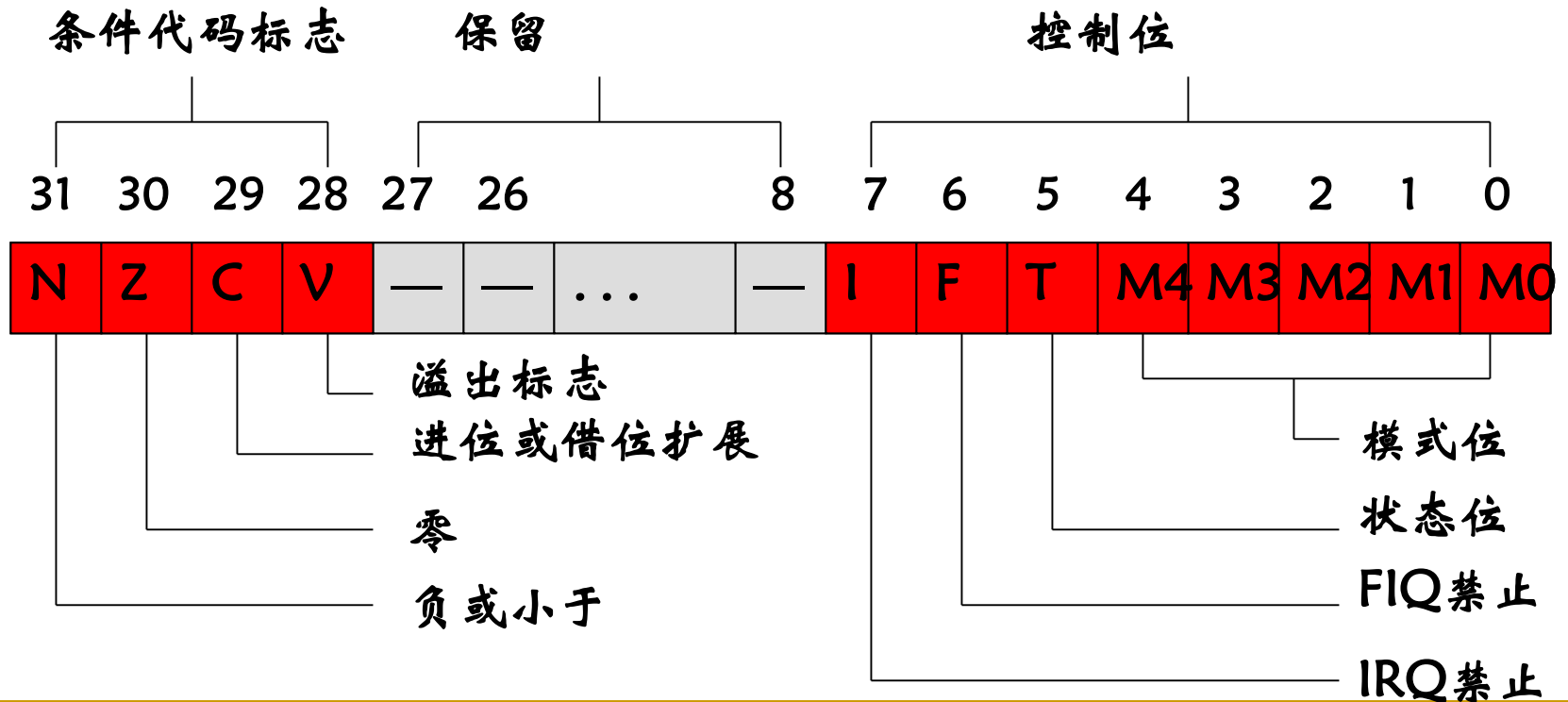
- 简介

ARM内核包含1个CPSR和5个仅供异常处理程序使用的SPSR。CPSR反映当前处理器的状态，其包含：

- 4个条件代码标志(负标志N、零标志Z、进位标志C和溢出标志V)；
- 2个中断禁止位(IRQ禁止与FIQ禁止)；
- 5个对当前处理器模式进行编码的位(M[4:0])；
- 1个用于指示当前执行指令的位(ARM指令还是Thumb指令)。

3.4 当前程序状态寄存器

• 程序状态寄存器的格式



3.4 当前程序状态寄存器

• 条件代码标志

各标志位的含义如下：

- 负标志N：运算结果的第31位值，记录标志设置操作的结果；
- 零标志Z：如果标志设置的操作为0，则置位；
- 进位标志C：记录无符号加法溢出，减法无借位，循环移位；
- 溢出标志V：记录标志设置操作的有符号溢出。



3.4 当前程序状态寄存器

• 控制位

3: 处理器模式位M0~M4。

M[4:0]	模式	可见的Thumb状态寄存器	可见的ARM状态寄存器
10000	用户	R0~R7, SP, LR, PC, CPSR	R0~R14, PC, CPSR
10001	快速中断	R0~R7, SP_fiq, LR_fiq, PC, CPSR, SPSR_fiq	R0~R7, R8_fiq~R14_fiq, PC, CPSR, SPSR_fiq
10010	中断	R0~R7, SP_irq, LR_irq, PC, CPSR, SPSR_irq	R0~R12, R13_irq, R14_irq, PC, CPSR, SPSR_irq
10011	管理	R0~R7, SP_svc, LR_svc, PC, CPSR, SPSR_svc	R0~R12, R13_svc, R14_svc, PC, CPSR, SPSR_svc
10111	中止	R0~R7, SP_abt, LR_abt, PC,	R0~R12, R13_abt, R14_abt, PC,

注意：不是所有模式位的组合都定义了有效的处理器模式，如果将非法值写入M[4:0]中，处理器将进入一个无法恢复的模式。

运行；
行。

3.4 当前程序状态寄存器

- 保留位

CPSR中的保留位被保留将来使用。当改变CPSR标志和控制位时，请确认没有改变这些保留位。

另外，请确保您的程序不依赖于包含特定值的保留位，因为将来的处理器可能会将这些位设置为1或者0。

3.5 ARM体系的异常

- 异常简介

只要正常的程序流被暂时中止，处理器就进入异常模式。例如在用户模式下执行程序时，当外设向处理器内核发出中断请求导致内核从用户模式切换到异常中断模式。

如果同时发生两个或更多异常，那么将按照固定的顺序来处理异常。

• 异常入口/出口汇总

下表所示为异常返回地址值以及退出异常处理程序所推荐使用的指令。

异常或入口	返回指令	返回地址
SWI	MOVS PC,R14_svc	R14
未定义的指令	MOVS PC,R14_und	R14
预取中止	SUBS PC,R14_abt,#4	R14-4
快速中断	SUBS PC,R14_fiq,#4	R14-4
中断	SUBS PC,R14_irq,#4	R14-4
数据中止	SUBS PC,R14_abt,#8	R14-8
复位	无	—

注意：“MOVS PC,R14_svc”是指在管理模式执行 MOVS PC,R14 指令，同样类似的指令还有“MOVS PC,R14_und”、“SUBS PC,R14_abt,#4”等。

3.5 ARM体系的异常

• 异常向量表

地址	异常	进入时的模式	进入时I的状态	进入时F的状态
0x00000000	复位	管理	禁止	禁止
0x00000004	未定义指令	未定义	I	F
0x00000008	软件中断异常	管理	禁止	F
0x0000000C	中止（预取）	中止	I	F
0x00000010	中止（数据）	中止	I	F
0x00000014	保留	保留	—	—
0x00000018	IRQ	中断	禁止	F
0x0000001C	FIQ	快速中断	禁止	禁止

注：表中的I和F表示不对该位有影响，保留原来的值。

3.5 ARM体系的异常

- 异常优先级

当多个异常同时发生时，一个固定的优先级决定系统处理它们的顺序。



优先级	异常
1	复位
2	数据中止
3	FIQ
4	IRQ
5	预取指中止
6	未定义指令中止
6	软件中断异常

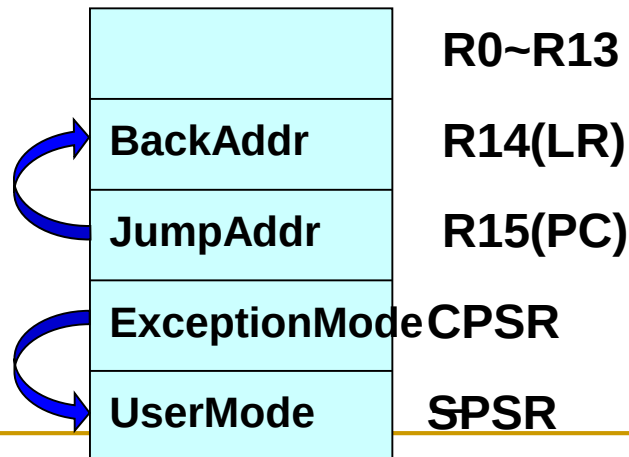
3.5 ARM体系的异常

• 异常的进入

当一程序代码或数据访问在用户模式有错的做如下处理：

- 将CPSR寄存器中的异常标志位（ELR）置位，并保存当前PC地址到BackAddr，然后跳转到异常处理程序执行；

异常模式



3.5 ARM体系的异常

• 异常的进入

当一个异常导致模式切换时，内核自动的做如下处理：

- 将异常处理程序的返回地址（加固定的偏移量）保存到相应异常模式下的LR；
- 将CPSR的当前值保存到相应异常模式下的SPSR；
- 设置CPSR为相应的异常模式；
- 设置PC为相应异常处理程序的中断入口向量地址，跳转到相应的异常中断处理程序执行；

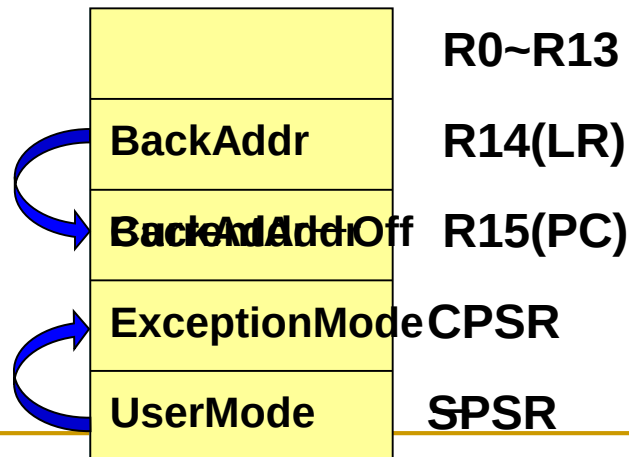
3.5 ARM体系的异常

• 异常的退出

当异常处理程序结束时，异常处理程序必须：

- 将跟异常值相关的PCSR指令的下一条指令处执行，即就是说将LR中的值减去偏移量后移入PC；

异常模式



3.5 ARM体系的异常

• 异常的退出

当异常处理程序结束时，异常处理程序必须：

- 返回到发生异常中断的指令的下一条指令处执行，即就是说将LR中的值减去偏移量后移入PC；
- 将SPSR的值复制回CPSR；
- 清零在入口处置位的中断禁止标志。

3.5 ARM体系的异常

• 复位异常

当nRESET信号被拉低时，ARM处理器放弃正在执行的指令，等到nRESET信号再次变高时，处理器执行一下操作：

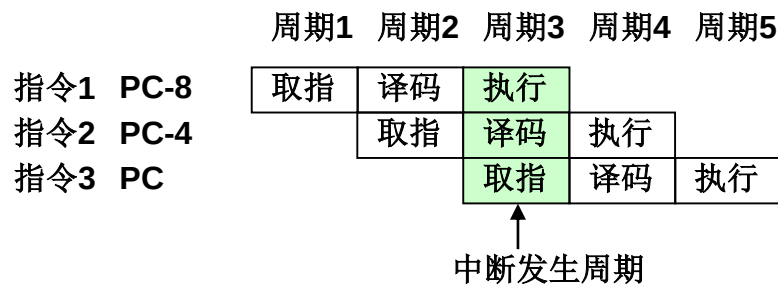
- 强制M[4:0]变为**b10011**，系统进入**管理模式**；
- 将CPSR中的标志位**I和F置位**，**IRQ与FIQ**中断被**禁止**；
- 将CPSR中的标志位**T清零**，处理器处于**ARM状态**；
- 强制**PC**从地址**0x00**开始对下一条指令进行取指；
- 返回到ARM状态并恢复执行。

3.5 ARM体系的异常

中断请求异常

只有当CPSR中相应的中断屏蔽位被清除时，才可能发生IRQ异常，中断请求 (IRQ) 异常由一个nIRQ输入端的低电平所产生的正常中断。

注：中断异常产生时，中断异常模式下的R14保存的是PC的值。



指令1的执行不会中断；

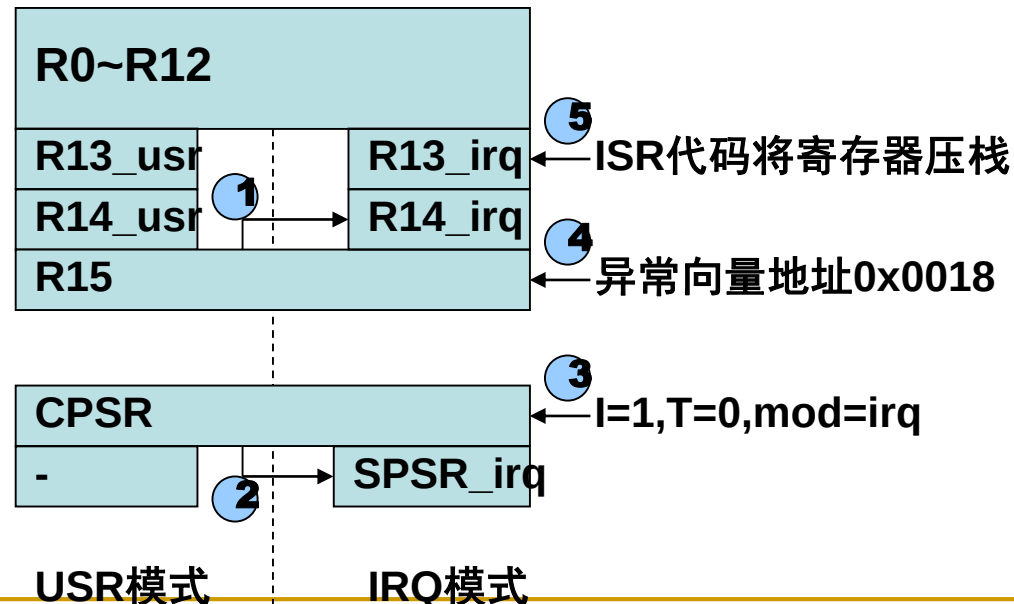
异常程序结束时返回到指令2；

进入中断服务程序时，指令3地址被保存在R14中。

3.5 ARM体系的异常

• 进入IRQ异常模式

5 当发生IRQ异常时，CPU将PC寄存器中的下一条指令地址保存到R13_usr寄存器中，然后将R13_irq寄存器设置为异常向量地址0x0018，开始执行异常处理程序。

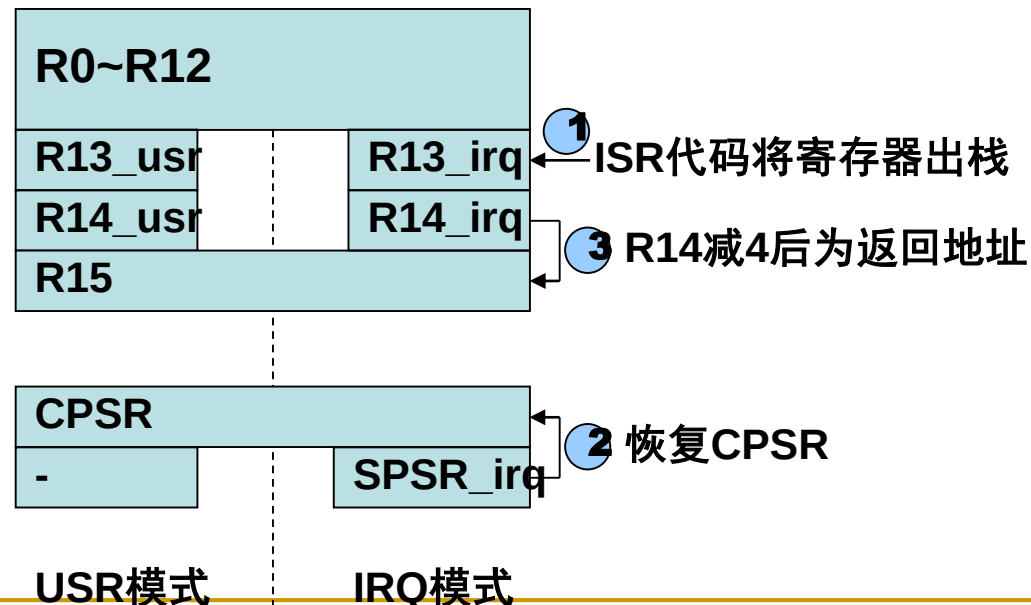


3.5 ARM体系的异常

退出IRQ异常模式

3 将PC寄存器减4，实现指令为：

SUBS PC,R14_irq,#4



3.5 ARM体系的异常

- 快速中断请求异常

快速中断请求(FIQ)适用于对一个突发事件的快速响应，这得益于在ARM状态中，快中断模式有8个专用的寄存器可用来满足寄存器保护的需要（这可以加速上下文切换的速度）。

不管异常入口是来自ARM状态还是Thumb状态，FIQ处理程序都会通过执行下面的指令从中断返回：

```
SUBS PC,R14_fiq,#4
```

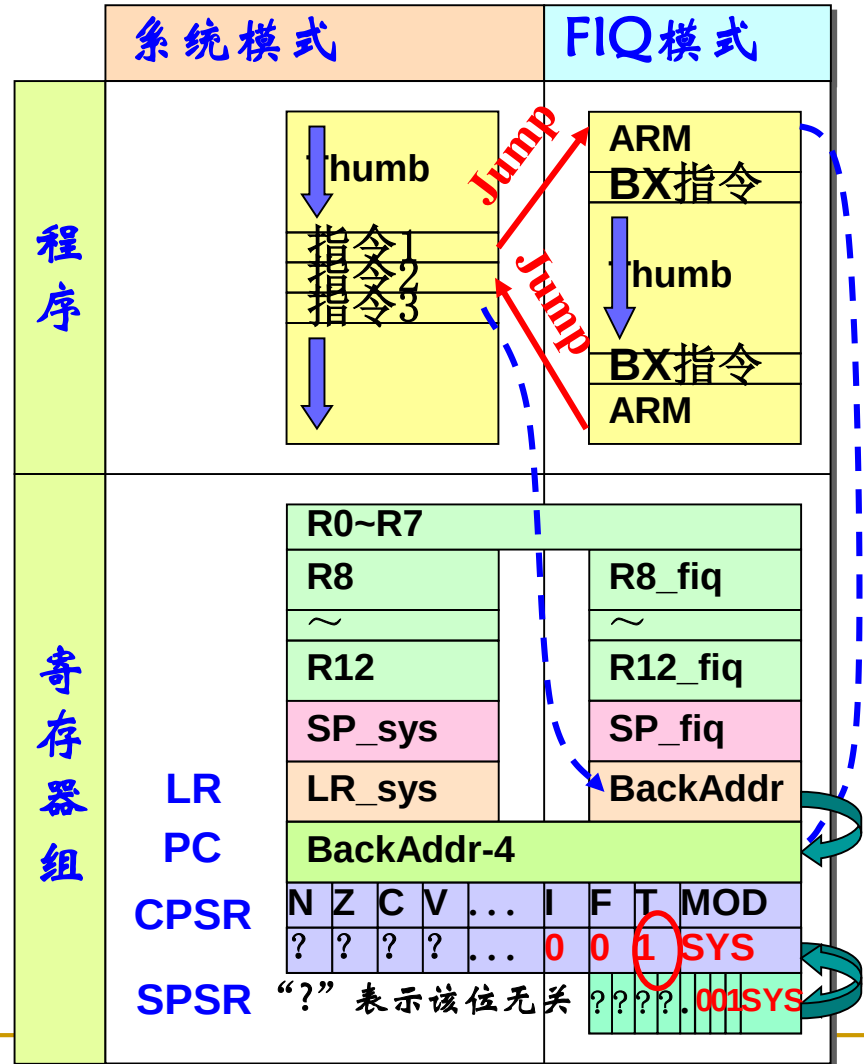
在一个特权模式中，可以通过置位CPSR中的F位来禁止FIQ异常。

FIQ中断的例子

FIQ中断发生时，处理器会自动切换到FIQ模式。在FIQ模式下，处理器会执行FIQ向量表中的指令。如果FIQ向量表中没有指令，处理器会执行默认的FIQ向量地址。在FIQ模式下，处理器会执行FIQ向量表中的指令，直到遇到一个分支指令（如BX或ARM）为止。此时，处理器会根据该指令的跳转目标，切换到指定的模式（如ARM或FIQ）。在FIQ模式下，处理器会执行FIQ向量表中的指令，直到遇到一个分支指令（如BX或ARM）为止。此时，处理器会根据该指令的跳转目标，切换到指定的模式（如ARM或FIQ）。在FIQ模式下，处理器会执行FIQ向量表中的指令，直到遇到一个分支指令（如BX或ARM）为止。此时，处理器会根据该指令的跳转目标，切换到指定的模式（如ARM或FIQ）。

（指令2的地址）。

· 设置MOD位，切换处理器模式至FIQ模式。



3.5 ARM体系的异常

- 未定义的指令异常

未定义指令异常是内部异常中断，当ARM处理器遇到一条自己和系统内部任何协处理器都无法执行的指令时，就会发生未定义指令异常，从而进入中断处理程序，同时软件可使用这一机制通过仿真未定义的协处理器指令来扩展ARM指令集。

在仿真失败的指令后，捕获处理器执行下面的指令：

MOVS PC,R14_und

3.5 ARM体系的异常

- 中止异常

中止表示当前对存储器的访问不能被完成，这是由外部ABORT输入信号引起的异常中断。中止类型有2种类型：

- 预取指中止：由程序存储器引起的中止异常；
- 数据中止：由数据存储器引起的中止异常。

3.5 ARM体系的异常

- 预取指中止

当发生预取指中止时，ARM内核将预取的指令标记为无效，但在指令到达流水线的执行阶段时才进入异常。如果指令在流水线中因为发生分支而没有被执行，中止将不会发生。

在处理中止的原因之后，不管处于哪种处理器操作状态，处理程序都会执行下面的指令恢复PC和CPSR并重试被中止的指令：

```
SUBS PC,R14_abt,#4
```

3.5 ARM体系的异常

- 数据中止

当发生数据中止异常时，异常会在“导致异常的指令”执行后的下一条指令发生。在这种情况下，理想的状况是进入数据中止异常的ISR，然后在内存中挑选出问题，再重新执行导致异常的指令。

在修复产生中止的原因后，不管处于哪种处理器操作状态，处理程序都必须执行下面的返回指令：

```
SUBS PC,R14_abt,#8
```

3.5 ARM体系的异常

- 软件中断异常

软件中断异常(SWI)用于进入管理模式，通常用于请求一个特定的管理函数。SWI处理程序通过执行下面的指令返回：

MOVS PC,R14_svc

这个动作恢复了PC和CPSR并返回到SWI之后的指令。SWI处理程序读取操作码以提取SWI函数编号。

3.5 ARM体系的异常

- 中断延迟

中断延迟即从外部中断请求信号发出到执行对应的中断服务程序ISR的第1条指令所需要的时间。

通过软件程序设计来缩短中断延迟的方法有：**中断优先级**和**中断嵌套**。

3.6 ARM体系的存储系统

简介

ARM处理器系统由处理器核、本地缓存和外部存储器组成，指存储数据并控制其访问。系统将外部总线信号和时序转换为处理器内部总线信号，使处理器可访问存储器中的数据。

ARM7的规范定义了局部总线的信号和时序。

ARM7
处理器核

存储器
管理器

各芯片生产厂商制定了自己的外部总线的信号和时序。

存储器

3.6 ARM体系的存储系统

- 地址空间

ARM结构使用单个平面的 2^{32} 个8位字节地址空间。地址空间可以看作是包含 2^{30} 个32位字，或 2^{31} 个16位半字。如果地址向上或向下溢出地址空间，通常会发生翻转。

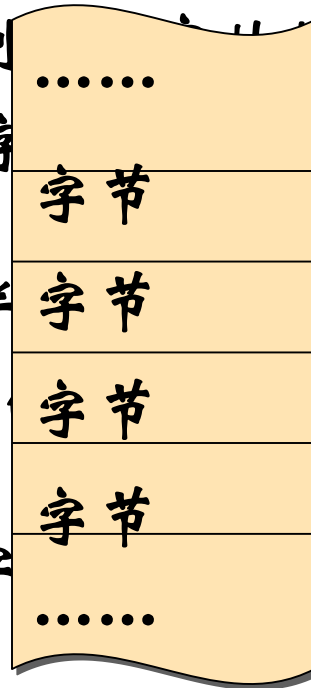
注意：如果在取指操作时地址发生溢出，只要没有执行预取的无效指令，就不会导致异常。

3.6 ARM体系的存储系统

• 存储器格式

地址空间的规则

- 位于地址A的字 (4字节) 位于地址A, A+1, A+2和A+3;
- 位于地址A的半字 (2字节) 位于地址A和A+1;
- 位于地址A+2的半字 (2字节) 位于地址A+2和A+3;
- 位于地址A的字 (4字节) 位于地址A和A+2;



存储器

2 大端存储
模式下，
A+3 0x12345678
放在高地址中。
A+2 在A地址开始
的一个字空间
A+1 的存储；
放在低地址中。

3.6 ARM体系的存储系统

• 非对齐的存储器访问

ARM结构通常希望所有的存储器访问都合理的对齐，具体来说就是字访问的地址通常是字对齐的，而半字访问使用的地址是半字对齐的。不按这种方式对齐的存储器访问称为**非对齐的存储器访问**。

- 将一个非字（半字）对齐的地址写入ARM（Thumb）状态的R15寄存器，将引起**非对齐的指令取指**。
- 在一个非字（半字）对齐的地址读写一个字（半字），将引起**非对齐的数据访问**：